Saviours AI

ICSE CLASS 10 · 2027 SYLLABUS

SIDE KICK

Computer Applications

Java for ICSE Class 10, written fresh for the 2027 syllabus: every concept, 50 must-know programs and exam-style practice with answers.



EVERY TOPPER HAS A SIDEKICK

saviours.pro

Hey. I'm your Sidekick.

I live inside this book. Wherever things get tricky, I pop up with a tip, a trap to dodge, or a question the board loves to ask. Read the chapter, then try my HOT questions. Deal?



Aligned to the official ICSE 2027 syllabus. If it's in the syllabus, it's in here.

SIDEKICK TIP
Shortcuts, mnemonics and exam hacks.

FUN FACT
Things that make the concept stick.

REAL LIFE
Where this shows up outside the exam hall.

KEY POINT
Definitions and laws you must write exactly.

HOT QUESTION
Higher-order thinking. Try before you peek.

✦ NEW
Added in this edition to match the 2027 syllabus.

Contents

01 Revision of Class IX Syllabus

02 Class as the Basis of All Computation

03 User-Defined Methods

04 Constructors

05 Library Classes

06 Encapsulation

07 Arrays

08 String Handling

09 50 Must-Know Programs

10 Practice Paper: Exam-Style Questions with Answers

11 Lab Companion: Assignments and Marking

01

CHAPTER · COMPUTER APPLICATIONS

Revision of Class IX Syllabus

Class X Java is built on Class IX Java. Before we climb higher, we rebuild the base: objects, data types, operators, input, Math methods, decisions and loops.



IN THIS CHAPTER

- OOP concepts
- Data types
- Operators
- Scanner input
- Math methods
- if / switch
- Loops
- Nested loops

Object Oriented Programming in a Nutshell

Java builds a program out of **objects**, the way a city is built out of buildings. Each object keeps its own data and knows the jobs it can do. Four ideas hold the whole style together.

CONCEPT	MEANING	EVERYDAY PICTURE
Abstraction	Show only what is needed, hide the messy details	You press a button on the microwave and never see the circuits
Encapsulation	Wrap data and the code that uses it into one unit (a class) and guard it	A medicine capsule holds powder inside one shell
Inheritance	A new class acquires the features of an existing class	A child inherits traits from parents
Polymorphism	One name, many forms	The word 'run' means different things for a person, a tap and a program



KEY POINT

Class = blueprint (a template). **Object** = a real thing made from that blueprint. One class can make many objects, just as one house plan can produce many houses.

Structure of a Java Program

```
import java.util.*;
public class CircleArea
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the radius: ");
        double r = sc.nextDouble();
        double area = Math.PI * r * r;
        System.out.println("Area = " + area);
    }
}
```

```
Input: 3
----------------------------
Enter the radius: Area = 28.274333882308138
```

- `import java.util.*;` brings in the `Scanner` class from the `java.util` package.
- The file name must match the **public class** name (`CircleArea.java`).
- Execution always begins at `public static void main(String[] args)`.
- Every statement ends with a semicolon. Blocks are wrapped in braces `{ }`.
- Text in double quotes is a **String**; the + beside a string joins (concatenates).

Tokens: The Building Blocks

A **token** is the smallest unit the compiler understands. Java has five kinds.

TOKEN	EXAMPLES
Keywords (reserved words)	class, public, static, void, int, if, else, while, for, return
Identifiers (names you choose)	total, area, StudentMarks, _x, $cost
Literals (fixed values)	25, 3.14, 'A', "hello", true
Operators	+ - * / % == && ++
Separators (punctuators)	( ) { } [ ] ; , .



BOARD ALERT

An identifier may use letters, digits, _ and $, must **not start with a digit**, cannot contain spaces and cannot be a keyword. Java is **case sensitive**: `Total` and `total` are two different names.

Data Types

A data type tells Java how much memory to reserve and what kind of value a variable may hold. **Primitive** types are built in and hold a single plain value.

TYPE	SIZE	HOLDS	EXAMPLE
byte	1 byte	Tiny whole numbers (-128 to 127)	byte b = 100;
short	2 bytes	Small whole numbers	short s = 5000;
int	4 bytes	Whole numbers (about +/- 2.1 billion)	int n = 45000;
long	8 bytes	Very large whole numbers	long pop = 7800000000L;
float	4 bytes	Decimals, about 7 digits accuracy	float f = 2.5f;
double	8 bytes	Decimals, about 15 digits accuracy	double d = 9.81;
char	2 bytes	A single character in single quotes	char c = 'K';
boolean	1 bit (logical)	Only true or false	boolean ok = true;

`String` is not primitive. It is a class and holds a sequence of characters in double quotes.

Type conversion

- **Implicit (automatic):** a smaller type widens to a larger one. `int x = 7; double d = x;` gives `7.0`.
- **Explicit (casting):** you force a larger type into a smaller one and may lose data. `double p = 9.99; int q = (int) p;` gives 9, the decimal part is chopped (not rounded).

- Order of widening: byte → short → int → long → float → double. A `char` widens to `int` (its Unicode value).



SIDEKICK TIP

Final variables. Add `final` to make a constant: `final double GST = 0.18;`. Changing it later is a compile error.

Operators

KIND	OPERATORS	NOTES
Arithmetic	+ - * / %	Integer division drops the decimal: 7/2 is 3. % gives the remainder: 7%2 is 1
Relational	< > <= >= == !=	Result is always boolean
Logical	&& || !	AND, OR, NOT. && and || stop early (short circuit)
Assignment	= += -= *= /= %=	x += 5 means x = x + 5
Increment/decrement	++ --	Prefix changes first, postfix uses first and then changes
Conditional (ternary)	cond ? a : b	Picks a if cond is true, else b

Watch the difference between prefix and postfix. This program is a favourite output-prediction question.

```
public class IncDemo
{
    public static void main(String[] args)
    {
        int a = 5;
        int b = a++ + ++a;
        System.out.println("a = " + a + ", b = " + b);
        int c = 10;
        c += c-- - --c;
        System.out.println("c = " + c);
    }
}
```

```
Output
------------------------------
a = 7, b = 12
c = 12
```

STEP	WORKING	VALUES
a++	Uses 5, then a becomes 6	a = 6, value used = 5
++a	a becomes 7 first, then used	a = 7, value used = 7
b	5 + 7	b = 12
c-- (inside)	uses 10, then c = 9	left c is 10 (saved)
--c	c becomes 8, used	8
Final	c = 10 + (10 - 8)	c = 12



BOARD ALERT

In `c += expr`, Java remembers the old value of `c` (10) before evaluating `expr`. That is why the answer is 12, not 10.

Operator precedence, high to low: `( )`, then `++ -- !`, then `* / %`, then `+ -`, then relational, then `== !=`, then `&&`, then `||`, then `?:`, then assignment.

Input in Java

Create a `Scanner` object tied to the keyboard (`System.in`), then call the method matching the data type.

METHOD	READS
nextInt()	an int
nextLong()	a long
nextFloat() / nextDouble()	a float / a double
next()	one word (stops at a space)
nextLine()	a whole line, spaces included
next().charAt(0)	a single character
nextBoolean()	true or false



BOARD ALERT

The nextLine() trap. After `nextInt()` the Enter key is still waiting in the buffer, so a following `nextLine()` returns an empty line. Fix: add an extra `sc.nextLine();` to swallow the leftover Enter, or read everything with `nextLine()` and convert.

Other ways to give input without Scanner: pass values as **command-line arguments** (`args[0]` is always a String), or assign values to variables inside the program.

Mathematical Library Methods

The `Math` class lives in `java.lang`, so no import is needed. All its methods are `static`: call them as `Math.name(...)`.

```
public class MathLab
{
    public static void main(String[] args)
    {
        System.out.println(Math.pow(2, 10));
        System.out.println(Math.sqrt(144));
        System.out.println(Math.cbrt(27));
        System.out.println(Math.abs(-7.5));
        System.out.println(Math.max(14, 9));
        System.out.println(Math.min(-3, -8));
        System.out.println(Math.ceil(4.1));
        System.out.println(Math.floor(-4.1));
        System.out.println(Math.round(4.5));
        System.out.println(Math.round(-4.5));
        System.out.println(Math.rint(2.5));
        System.out.println(Math.rint(3.5));
        System.out.println((int) (Math.random() * 6) + 1 ⩽ 6);
    }
}
```

```
Output
------------------------------
1024.0
12.0
3.0
7.5
14
-8
5.0
-5.0
5
-4
2.0
4.0
true
```

METHOD	RETURNS	REMEMBER
Math.pow(x, y)	double	x raised to the power y
Math.sqrt(x) / Math.cbrt(x)	double	square root / cube root
Math.abs(x)	same type as x	removes the sign
Math.max(a,b) / Math.min(a,b)	type of the larger input type	bigger / smaller of two
Math.ceil(x)	double	smallest whole number ≥ x
Math.floor(x)	double	largest whole number ≤ x
Math.round(x)	long (for double), int (for float)	nearest integer; .5 goes up (toward +infinity)
Math.rint(x)	double	nearest integer; exact .5 goes to the **even** neighbour
Math.random()	double	number in [0, 1), never reaches 1



HOT QUESTION

Write a Java expression to generate a random whole number between 10 and 20 (both included). **Hint:** `(int)(Math.random() * 11) + 10`. There are 11 numbers from 10 to 20, so multiply by 11 and add the starting value.

Conditional Constructs

if, if-else and if-else-if ladder

```
import java.util.*;
public class GradeFinder
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Marks out of 100: ");
        int m = sc.nextInt();
        char g;
        if (m < 0 || m > 100)
            System.out.println("Invalid marks");
        else
        {
            if (m ⩾ 90)      g = 'A';
            else if (m ⩾ 75) g = 'B';
            else if (m ⩾ 50) g = 'C';
            else             g = 'D';
            System.out.println("Grade: " + g);
        }
    }
}
```

```
Input: 82
-----------------------------
Marks out of 100: Grade: B
```

switch

`switch` compares one value (int, char, String) with fixed `case` labels. Without `break`, execution **falls through** into the next case. `default` runs when nothing matches.

```
public class DayName
{
    public static void main(String[] args)
    {
        int d = 3;
        switch (d)
        {
            case 1:  System.out.println("Mon"); break;
            case 2:  System.out.println("Tue"); break;
            case 3:  System.out.println("Wed");
            case 4:  System.out.println("Thu"); break;
            default: System.out.println("Other");
        }
    }
}
```

```
Output
-----------------------------
Wed
Thu
```



FUN FACT

Case 3 has no `break`, so the output shows both Wed and Thu. This 'fall through' is the most common trick in board output questions.

USE IF-ELSE-IF WHEN	USE SWITCH WHEN
Conditions are ranges or use relational/logical operators (marks >= 50)	One variable is compared with exact constants (1, 2, 3 or 'A', 'B')
Different variables are tested	Menu-driven programs with options 1, 2, 3 ...

The ternary operator

```
public class TernaryDemo
{
    public static void main(String[] args)
    {
        int a = 17, b = 24;
        int big = (a > b) ? a : b;
        String kind = (big % 2 == 0) ? "even" : "odd";
        System.out.println(big + " is " + kind);
    }
}
```

```
Output
-----------------------------
24 is even
```

Iterative Constructs (Loops)

A loop repeats a block while a condition stays true. Three loops, three situations.

LOOP	SYNTAX	CHECKS CONDITION	USE WHEN
for	for (init; test; update)	Before each pass (entry controlled)	Number of repetitions is known
while	while (test)	Before each pass (entry controlled)	Repetitions depend on a condition
do-while	do { } while (test);	After each pass (exit controlled)	Body must run **at least once**

```
public class LoopTrio
{
    public static void main(String[] args)
    {
        for (int i = 1; i ⩽ 5; i++)
            System.out.print(i * i + " ");
        System.out.println();
        int n = 1234, rev = 0;
        while (n > 0)
        {
            rev = rev * 10 + n % 10;
            n = n / 10;
        }
        System.out.println(rev);
        int k = 10;
        do
        {
            System.out.println("k = " + k);
            k++;
        } while (k < 10);
    }
}
```

```
Output
------------------------------
1 4 9 16 25
4321
k = 10
```

PASS	N (START)	N % 10	REV	N AFTER N/10
1	1234	4	0*10+4 = 4	123
2	123	3	4*10+3 = 43	12
3	12	2	43*10+2 = 432	1
4	1	1	432*10+1 = 4321	0 (loop ends)



KEY POINT

The **do-while** printed `k = 10` even though `k < 10` is false at the start. The body runs first and the test comes afterwards.

break leaves the loop completely. **continue** skips the rest of this pass and goes to the next one. A loop such as `for (;;)` or `while (true)` is infinite unless a `break` or `return` stops it.

Nested for Loops

A loop inside a loop. For every single pass of the **outer** loop, the **inner** loop runs from start to finish. Outer loops usually control rows; inner loops control columns.

```
public class StarTriangle
{
    public static void main(String[] args)
    {
        for (int i = 1; i ⩽ 4; i++)
        {
            for (int j = 1; j ⩽ i; j++)
                System.out.print("* ");
            System.out.println();
        }
        System.out.println();
        for (int i = 1; i ⩽ 4; i++)
        {
            for (int s = 1; s ⩽ 4 - i; s++)
                System.out.print(" ");
            for (int j = 1; j ⩽ i; j++)
                System.out.print(j);
            System.out.println();
        }
    }
}
```

```
Output
------------------------------
*
* *
* * *
* * * *

   1
  12
 123
1234
```



SIDEKICK TIP

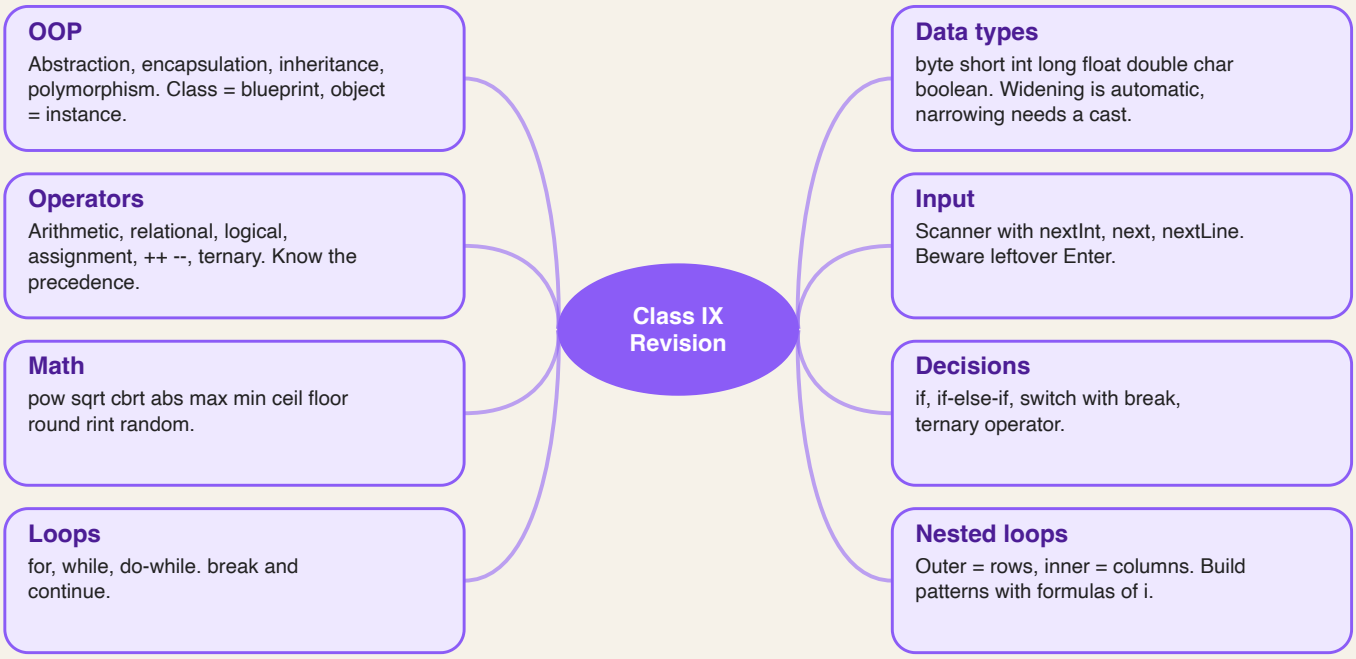
Pattern recipe. (1) Count the rows: that is the outer loop. (2) For each row, find how many spaces and how many symbols appear. (3) Write each as a formula in terms of the row number `i`. (4) End each row with `println()`.



REAL LIFE

Nested loops are how screens draw pixels (rows then columns), how spreadsheets visit every cell, and how games scan a board.

Mind Map Recap



HOT Questions

HOT QUESTION

Predict the output: `int x = 8; x = x++ * 2 + x--; System.out.println(x);` (Answer: 8*2 is 16 with x now 9, then + 9 gives 25.)

HOT QUESTION

Why does `System.out.println(7 / 2 * 2.0);` print `6.0` and not `7.0`? (Answer: `7/2` is integer division = 3, then 3 * 2.0 = 6.0.)

HOT QUESTION

Which loop would you choose to keep asking a user for a password until it is correct, and why? (Answer: do-while, because the question must be asked at least once.)

02

CHAPTER · COMPUTER APPLICATIONS

Class as the Basis of All Computation

Everything around you is an object with features and abilities. A Java class is the blueprint that captures both. This chapter shows how real things become code.



IN THIS CHAPTER

- Objects: state and behaviour
- Class as a factory
- Primitive vs composite
- Creating objects
- Reference vs value

Objects: State and Behaviour

Look at a mobile phone. It has **features** (brand, colour, battery level, price) and it can **do** things (call, click a photo, play music). Java models any real or imaginary thing the same way.

OBJECT	STATE (DATA / ATTRIBUTES)	BEHAVIOUR (METHODS)
Bank account	accountNo, holderName, balance	deposit(), withdraw(), showBalance()
Student	name, rollNo, marks	calcTotal(), grade(), display()
Traffic light	colour, timer	changeColour(), showColour()
Pen	brand, inkColour, price	write(), refill()



KEY POINT

State = values stored in **member variables** (also called data members, attributes or fields). **Behaviour** = operations written as **member methods** (functions or messages). Variables define the state; methods define the behaviour.

Class: An Object Factory

A **class** is a user-defined data type that groups member variables and member methods. It describes *what* every object of that kind will hold and do. It is an **abstraction** for a whole set of similar objects, and acts as a **factory**: you can stamp out as many objects as you like.

```
class Student               // the blueprint
{
    String name;            // member variables (state)
    int roll;
    double marks;

    void show()             // member method (behaviour)
    {
        System.out.println(name + " " + roll + " " + marks);
    }
}
```



REAL LIFE

Class `Car` is the design drawing in the company's office. A particular red Car with number plate XY-04 parked outside is an **object** (an **instance** of the class).

Creating and Using Objects

An object is created with the new operator. The dot operator (`.`) reaches its members.

```
public class Student
{
    String name;
    int roll;
    double marks;

    void show()
    {
        System.out.println(name + " | " + roll + " | " + marks);
    }

    public static void main(String[] args)
    {
        Student s1 = new Student();
        Student s2 = new Student();
        s1.name = "Aarav";  s1.roll = 11;  s1.marks = 88.5;
        s2.name = "Diya";   s2.roll = 12;  s2.marks = 91.0;
        s1.show();
        s2.show();
    }
}
```

```
Output
-----------------------------
Aarav | 11 | 88.5
Diya | 12 | 91.0
```

1. Declaration
`Student s1;` creates only a **reference variable** (a name tag). No object exists yet; s1 holds `null`.

2. Instantiation
`new Student()` allocates memory for the object on the **heap** and sets default values (0, 0.0, null, false).

3. Assignment
`s1 = new Student();` stores the object's address in the reference variable s1.

4. Access
`s1.name` or `s1.show()` uses the dot to reach members of that one object.



SIDEKICK TIP

Each object has its **own copy** of the instance variables. Changing `s1.marks` does not touch `s2.marks`.

DEFAULT VALUES OF INSTANCE VARIABLES	VALUE
byte, short, int, long	0
float, double	0.0
char	'\u0000' (the null character)
boolean	false
Any class type (String, arrays, objects)	null

Primitive vs Composite Data Types

POINT	PRIMITIVE TYPE	COMPOSITE (CLASS / REFERENCE) TYPE
Examples	int, double, char, boolean	String, Scanner, arrays, Student
Defined by	Java itself	Java library or the programmer
What the variable holds	The actual value	The **address** (reference) of an object
Memory	Stack, fixed size	Object on the heap; reference on the stack
Default value	0, 0.0, false …	null
Can have methods?	No	Yes, members can be reached with the dot
Copy with =	Copies the value	Copies the address, so two names point to one object

A class lets you invent a **new data type**. Once `Student` exists, you can declare variables of type Student exactly as you would with `int`. This is why a class is called a user-defined or composite type.

Value Copy vs Reference Copy

```
public class CopyDemo
{
    int v;

    public static void main(String[] args)
    {
        int a = 10;
        int b = a;
        b = 99;
        System.out.println(a + " " + b);

        CopyDemo p = new CopyDemo();
        p.v = 10;
        CopyDemo q = p;
        q.v = 99;
        System.out.println(p.v + " " + q.v);
    }
}
```

```
Output
------------------------------
10 99
99 99
```

LINE	A	B	P.V	Q.V	NOTE
int b = a;	10	10			Two separate boxes
b = 99;	10	99			Only b changed
q = p;			10	10	One object, two names
q.v = 99;			99	99	Both names see the change



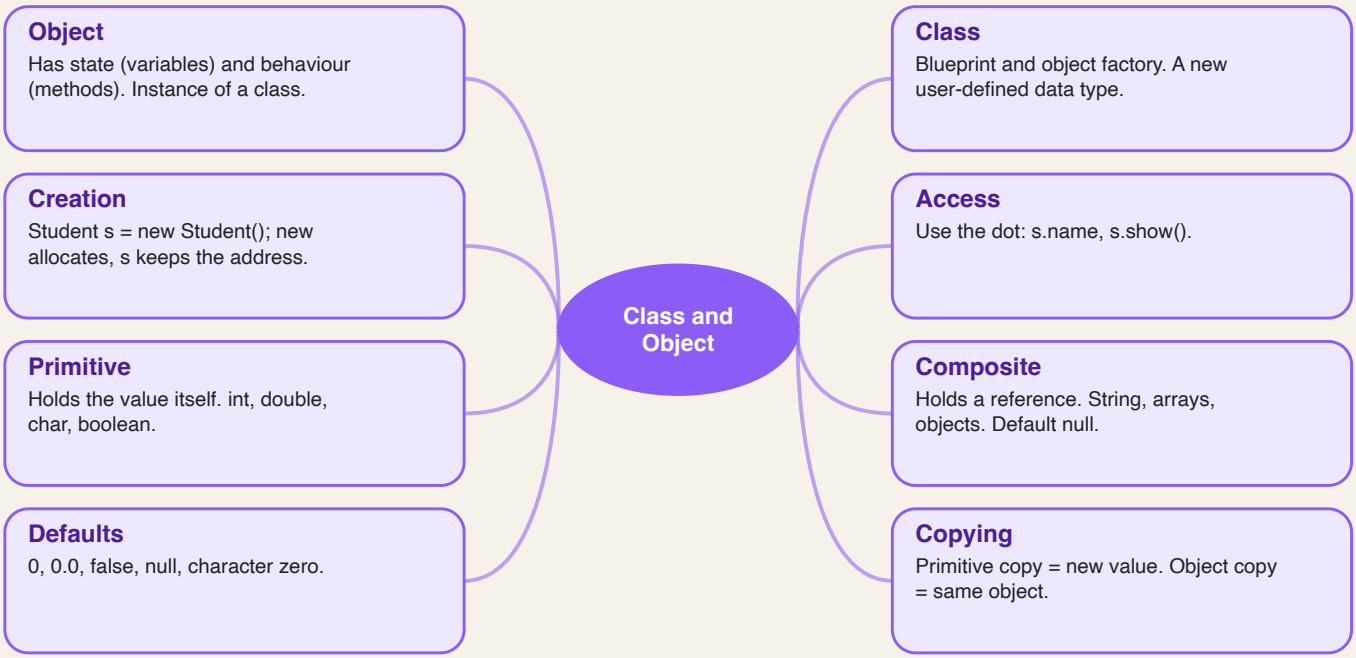
BOARD ALERT

q = p does **not** clone the object. Both references point to the same object, so a change through one is visible through the other.

Class vs Object at a Glance

CLASS	OBJECT
A template or blueprint	A real instance built from the template
Logical entity; occupies no memory for instance data	Physical entity; occupies memory
Declared once with the `class` keyword	Many can be made using `new`
Example: Student	Example: s1, s2

Mind Map Recap



HOT Questions

HOT QUESTION

A class has no objects yet. Does it occupy memory for its instance variables? (Answer: no. Memory is allotted only when an object is created with new.)

HOT QUESTION

Give one example each of an object whose state changes with time and whose behaviour changes its state. (Answer: a bank account: balance is state, `withdraw()` changes it.)

HOT QUESTION

`String a = new String("hi"); String b = a;` How many String objects exist? (Answer: one, with two references.)

03

CHAPTER · COMPUTER APPLICATIONS

User-Defined Methods

A method is a named mini-program inside your class. Write it once, call it many times. Methods are how you break a big problem into small, testable pieces.



IN THIS CHAPTER

- Why methods
- Syntax and signature
- Four forms
- Static vs non-static
- Overloading
- Pure vs impure
- Call by value / reference

Why We Need Methods

- **Reusability:** write once, call from anywhere.
- **Modularity:** split a big task into small named parts (modular programming), each easy to test.
- **Readability:** `area()` tells you more than ten lines of arithmetic.
- **Easy changes:** fix a mistake in one place, not in ten.

Syntax of a Method

```
accessSpecifier  [static]  returnType  methodName ( parameterList )
{
    // body: statements
    return value;      // required unless returnType is void
}
```

PART	MEANING	EXAMPLE
Access specifier	Who may call it: public, private, protected	public
static (optional)	Belongs to the class, not to an object	static
Return type	Type of the value sent back, or `void` if nothing	int
Method name	An identifier; start with a lowercase verb	findMax
Parameter list	Typed variables that receive the data (can be empty)	(int a, int b)

The **method prototype** (also called **signature** or **declaration**) is the top line without the body: the return type, name and parameter types, such as `int findMax(int a, int b)`. In Java the method **signature** strictly means the name plus parameter types, which is what overloading depends on.

Method Definition and Method Call

```
public class Calc
{
    int cube(int n)                 // definition
    {
        return n * n * n;
    }

    public static void main(String[] args)
    {
        Calc obj = new Calc();      // object needed: cube is non-static
        int r = obj.cube(4);        // method call
        System.out.println("Cube = " + r);
    }
}
```

```
Output
----------------------------
Cube = 64
```

TERM	MEANING	IN THE PROGRAM ABOVE
Formal parameter	Variable in the method *definition* that receives data	n
Actual parameter (argument)	Value or variable given in the method *call*	4
Return value	Result sent back with `return`	64, stored in `r`



KEY POINT

The number, order and data types of actual parameters must match the formal parameters. A `return` sends control back to the caller immediately; code after it in the same path is unreachable.

Four Forms of Methods

Methods differ by whether they take inputs and whether they give back an answer.

```
public class Forms
{
    void hello()                        // no input, no output
    {
        System.out.println("Hello!");
    }
    void square(int n)                  // input, no output
    {
        System.out.println(n * n);
    }
    int answer()                        // no input, output
    {
        return 42;
    }
    double avg(int a, int b)            // input and output
    {
        return (a + b) / 2.0;
    }
    public static void main(String[] args)
    {
        Forms f = new Forms();
        f.hello();
        f.square(9);
        System.out.println(f.answer());
        System.out.println(f.avg(7, 10));
    }
}
```

```
Output
----------------------------
Hello!
81
42
8.5
```

FORM	PARAMETERS	RETURN TYPE	TYPICAL USE
1	none	void	Print a menu or banner
2	yes	void	Display a computed result
3	none	a type	Read or generate a value
4	yes	a type	Calculate and give back (most useful)

Static and Non-Static Methods

POINT	STATIC METHOD	NON-STATIC (INSTANCE) METHOD
Keyword	Declared with `static`	No `static`
Belongs to	The class	An object of the class
Called by	`ClassName.method()`, or just `method()` inside the same class	`object.method()` only
Can use	Only static members directly	Static and instance members
Object needed?	No	Yes
Example	`Math.sqrt(9)`	`sc.nextInt()`

```
public class StaticDemo
{
    static int twice(int x)
    {
        return 2 * x;
    }
    int thrice(int x)
    {
        return 3 * x;
    }
    public static void main(String[] args)
    {
        System.out.println(twice(5));                // static: direct call
        System.out.println(StaticDemo.twice(6));     // via class name
        StaticDemo d = new StaticDemo();
        System.out.println(d.thrice(5));             // needs an object
    }
}
```

```
Output
------------------------------
10
12
15
```



BOARD ALERT

`main` is static, so calling a non-static method straight from it (`thrice(5);`) gives the error *non-static method cannot be referenced from a static context*. Create an object first.

Pure and Impure Methods

- A **pure method** does not change the state of the object. It only reads data and returns a result. Same input always gives the same output.
- An **impure method** changes the state (the instance variables) of the object, or anything outside itself. It is also called a **mutator**.

```
public class Wallet
{
    int cash = 100;

    int doubled()                  // pure: reads, does not change cash
    {
        return cash * 2;
    }
    void spend(int amt)            // impure: modifies cash
    {
        cash = cash - amt;
    }
    public static void main(String[] args)
    {
        Wallet w = new Wallet();
        System.out.println(w.doubled());
        System.out.println(w.cash);
        w.spend(30);
        System.out.println(w.cash);
    }
}
```

```
Output
-----------------------------
200
100
70
```

Method Overloading (Polymorphism)

Several methods may share **one name** in the same class if their parameter lists differ in the **number**, the **type** or the **order** of parameters. Java decides which one to run at compile time by matching the arguments. This is compile-time polymorphism.

```
public class Area
{
    double area(double r)                  // circle
    {
        return 3.14 * r * r;
    }
    int area(int l, int b)                 // rectangle
    {
        return l * b;
    }
    double area(double b, double h)        // triangle
    {
        return 0.5 * b * h;
    }
    public static void main(String[] args)
    {
        Area a = new Area();
        System.out.println(a.area(2.0));
        System.out.println(a.area(4, 5));
        System.out.println(a.area(4.0, 5.0));
    }
}
```

```
Output
------------------------------
12.56
20
10.0
```



BOARD ALERT

Only a different **return type** is **not** enough to overload. `int f(int a)` and `double f(int a)` in one class is a compile error. If no exact match exists, Java widens the argument (an int can go into a double parameter).

Call by Value

When you pass a **primitive** value, the method receives a **copy**. Whatever the method does to its formal parameter never reaches the original variable.

```
public class SwapTry
{
    void swap(int a, int b)
    {
        int t = a;
        a = b;
        b = t;
        System.out.println("inside: " + a + " " + b);
    }
    public static void main(String[] args)
    {
        SwapTry o = new SwapTry();
        int x = 3, y = 8;
        o.swap(x, y);
        System.out.println("outside: " + x + " " + y);
    }
}
```

```
Output
-----------------------------
inside: 8 3
outside: 3 8
```

MOMENT	X	Y	A	B
Before call	3	8	-	-
Copied into method	3	8	3	8
After swapping inside	3	8	8	3
After return	3	8	gone	gone

Call by Reference

When an **object or array** is passed, the method receives a copy of the **reference**, so it works on the **same object**. Changes made inside are visible outside. (Strictly, Java passes the reference itself by value, but the syllabus asks only for this definition with an example.)

```
public class RefDemo
{
    int n;

    void addTen(RefDemo p)
    {
        p.n = p.n + 10;
    }
    void zeroFirst(int[] arr)
    {
        arr[0] = 0;
    }
    public static void main(String[] args)
    {
        RefDemo r = new RefDemo();
        r.n = 5;
        r.addTen(r);
        System.out.println(r.n);
        int[] data = {7, 8, 9};
        r.zeroFirst(data);
        System.out.println(data[0] + " " + data[1] + " " + data[2]);
    }
}
```

```
Output
------------------------------
15
0 8 9
```

CALL BY VALUE	CALL BY REFERENCE
Copy of the value is passed	Address of the object or array is passed
Used for primitive types	Used for objects and arrays
Original data is **safe**	Original data can be **changed**

Modular Programming: Many Methods Working Together

Give each method one job and let `main` coordinate. The program below tests a number for being prime and finds the next prime, using three small methods.

```
import java.util.*;
public class PrimeHelper
{
    boolean isPrime(int n)
    {
        if (n < 2) return false;
        for (int i = 2; i * i ⩽ n; i++)
            if (n % i == 0) return false;
        return true;
    }
    int nextPrime(int n)
    {
        int k = n + 1;
        while (!isPrime(k))
            k++;
        return k;
    }
    void report(int n)
    {
        System.out.println(n + (isPrime(n) ? " is prime" : " is not prime"));
        System.out.println("Next prime after " + n + " is " + nextPrime(n));
    }
    public static void main(String[] args)
    {
        PrimeHelper p = new PrimeHelper();
        Scanner sc = new Scanner(System.in);
        p.report(sc.nextInt());
    }
}
```

```
Input: 90
-----------------------------
90 is not prime
Next prime after 90 is 97
```



SIDEKICK TIP

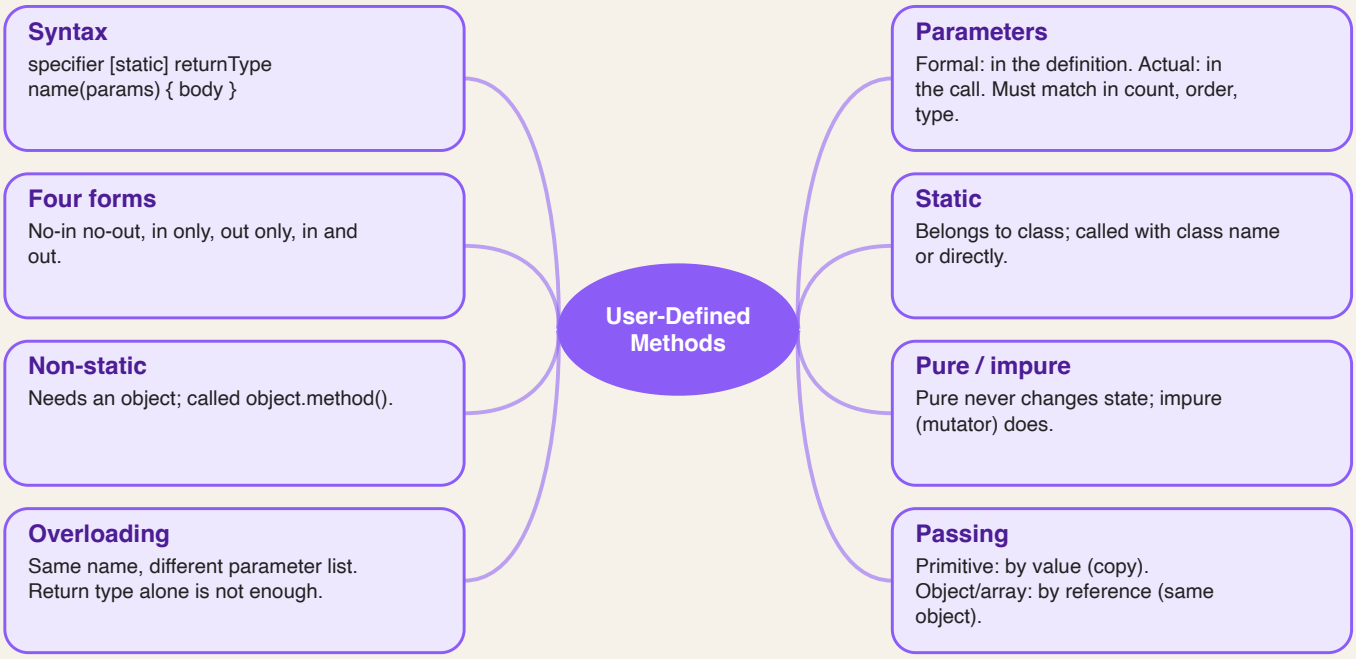
Notice `isPrime` is called from inside other methods without an object name. Methods of the same object can call each other directly by name.



FUN FACT

A method that calls itself is called **recursive**. It is outside the ICSE syllabus, so you will not be asked, but it is how a factorial can be written in two lines.

Mind Map Recap



HOT Questions

HOT QUESTION

A method has the prototype `static int f(int a)`. Write two valid overloads and one that is invalid. (Valid: `f(int a, int b)`, `f(double a)`. Invalid: `double f(int a)`.)

HOT QUESTION

Why can `main` call a static method directly but not a non-static one? (Answer: static members exist before any object; non-static need an object, and main has none until you create it.)

HOT QUESTION

A method receives an int array and sorts it. After the call the caller's array is sorted too. Explain. (Answer: arrays are passed by reference; both names point to the same array.)

04

CHAPTER · COMPUTER APPLICATIONS

Constructors

When a new object is born it needs its first values. A constructor is the special method that runs automatically at birth and sets everything up.



IN THIS CHAPTER

- Definition
- Characteristics
- Default
- Parameterised
- Overloading
- Constructor vs method

What Is a Constructor?

A **constructor** is a member method that has the **same name as its class** and is executed **automatically** when an object is created with new. Its job is to initialise the instance variables of the new object.

```
class Book
{
    String title;
    int pages;

    Book()                  // constructor: same name, no return type
    {
        title = "Untitled";
        pages = 0;
    }
}
```

Characteristics

- Its name is **exactly the class name** (same spelling and capitals).
- It has **no return type**, not even `void`. Writing `void` turns it into an ordinary method.
- It is called **automatically** once per object, at creation. You cannot call it later with the dot.
- It is normally `public` (or at least not private) so that other classes can create objects.
- It can take parameters and can be **overloaded**.
- It cannot be `static`, `final` or abstract and cannot return a value.



BOARD ALERT

`void Book() { }` is **not** a constructor. It is just a method that happens to share the class name. The compiler will then insert a default constructor, and `new Book()` will not run your code.

Types of Constructors

1. Default (non-parameterised) constructor

Takes no arguments. If you write **no** constructor at all, the compiler supplies an invisible default one that sets every variable to its default value (0, 0.0, false, null). The moment you write any constructor yourself, that free one disappears.

```
public class BookDefault
{
    String title;
    int pages;

    BookDefault()
    {
        title = "Untitled";
        pages = 100;
        System.out.println("Constructor ran");
    }
    void show()
    {
        System.out.println(title + ", " + pages + " pages");
    }
    public static void main(String[] args)
    {
        BookDefault b = new BookDefault();
        b.show();
    }
}
```

```
Output
------------------------------
Constructor ran
Untitled, 100 pages
```

2. Parameterised constructor

Takes arguments so each object can start with **its own values**. Use `this` to separate the instance variable from the parameter with the same name.

```
public class Rectangle
{
    int length, breadth;

    Rectangle(int length, int breadth)
    {
        this.length = length;       // this.length = instance variable
        this.breadth = breadth;     // breadth = parameter
    }
    int area()
    {
        return length * breadth;
    }
    public static void main(String[] args)
    {
        Rectangle r1 = new Rectangle(5, 3);
        Rectangle r2 = new Rectangle(8, 2);
        System.out.println(r1.area() + " " + r2.area());
    }
}
```

```
Output
-----------------------------
15 16
```



KEY POINT

`this` is a reference to the **current object**. `this.length = length;` means: copy the parameter `length` into the *object's* variable `length`. Without `this` the statement would assign the parameter to itself and the object's variable would stay 0.

You may also use different parameter names (`Rectangle(int l, int b)`) and skip `this` altogether. Board answers often do exactly that.

Constructor Overloading

A class may hold **several constructors** with different parameter lists. Java picks the one matching the arguments in new. It is polymorphism applied to constructors.

```
public class Box
{
    double w, h, d;

    Box()                              // no arguments
    {
        w = h = d = 1;
    }
    Box(double side)                   // a cube
    {
        w = h = d = side;
    }
    Box(double w, double h, double d)  // a cuboid
    {
        this.w = w;
        this.h = h;
        this.d = d;
    }
    double volume()
    {
        return w * h * d;
    }
    public static void main(String[] args)
    {
        Box a = new Box();
        Box b = new Box(3);
        Box c = new Box(2, 3, 4.5);
        System.out.println(a.volume());
        System.out.println(b.volume());
        System.out.println(c.volume());
    }
}
```

```
Output
----------------------------
1.0
27.0
27.0
```

STATEMENT	CONSTRUCTOR CHOSEN	RESULT
new Box()	Box()	1 × 1 × 1 = 1.0
new Box(3)	Box(double)	int 3 widens to 3.0, so 27.0
new Box(2, 3, 4.5)	Box(double, double, double)	2 × 3 × 4.5 = 27.0

Order of Events when an Object Is Created

1. Memory
new reserves memory and fills instance variables with default values.

2. Constructor
The matching constructor body runs and gives the variables their real starting values.

3. Reference
The address of the finished object is stored in the reference variable.

```
public class Trace
{
    int x = 5;                    // initialiser runs first

    Trace()
    {
        System.out.println("Before: x = " + x);
        x = 20;
        System.out.println("After: x = " + x);
    }
    public static void main(String[] args)
    {
        System.out.println("Start");
        Trace t = new Trace();
        System.out.println("End: " + t.x);
    }
}
```

```
Output
----------------------------
Start
Before: x = 5
After: x = 20
End: 20
```

Difference Between a Constructor and a Method

POINT	CONSTRUCTOR	METHOD
Name	Same as the class name	Any valid identifier, usually a verb
Return type	None, not even void	Must have one, or void
Call	Automatic at new	Explicitly by the programmer
Purpose	Initialise the object	Perform an operation
How many times per object	Once	Any number of times
Inherited	No	Yes
Default provided by compiler	Yes, if none written	No



SIDEKICK TIP

Quick check in the exam: if the line has the class name and **no return type**, it is a constructor. If you see `void`, `int` etc. before the name, it is a method.

Spot the Output: Overloaded Constructor Trap

```
public class Pair
{
    int a, b;

    Pair()
    {
        a = 4;
        b = 6;
    }
    Pair(int x)
    {
        a = x;
        b = x * 2;
    }
    Pair(int x, int y)
    {
        a = y;
        b = x;
    }
    public static void main(String[] args)
    {
        Pair p = new Pair();
        Pair q = new Pair(5);
        Pair r = new Pair(7, 9);
        System.out.println(p.a + p.b);
        System.out.println(q.a + " " + q.b);
        System.out.println(r.a + "" + r.b);
    }
}
```

```
Output
------------------------------
10
5 10
97
```

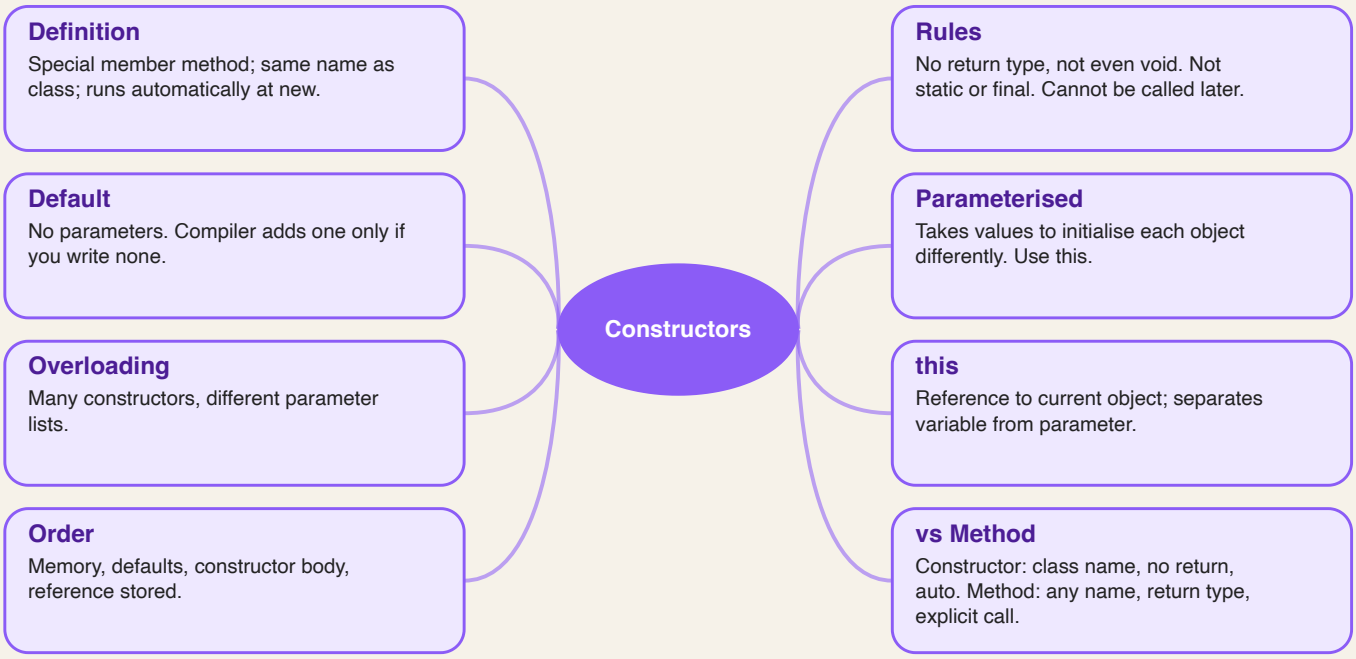
OBJECT	CONSTRUCTOR	A	B	PRINTED
p	Pair()	4	6	10 (sum of ints)
q	Pair(int)	5	10	5 10
r	Pair(int, int)	9	7	97 (empty string forces concatenation)



REAL LIFE

A game character can be made with new `Player()` (default name, health 100), new `Player("Ria")` or new `Player("Ria", 250)`. Overloaded constructors give players different starting setups with one class.

Mind Map Recap



HOT Questions

HOT QUESTION

A class has only `Pen(String colour)`. What happens when you write `new Pen()`? (Answer: compile error. Once you write a constructor the free default one is gone, so a no-argument constructor must be written by you.)

HOT QUESTION

Can a constructor be `private`? (Answer: yes, it compiles, but objects can then be created only inside the same class. Within the ICSE syllabus constructors are written public or default.)

HOT QUESTION

Is it possible for a class to have two constructors `A(int x, double y)` and `A(double y, int x)`? (Answer: yes, the order of types differs, so they are valid overloads.)

05

CHAPTER · COMPUTER APPLICATIONS

Library Classes

Java ships with a giant toolbox of ready-made classes. In Class X you meet the wrapper classes, which turn plain values into objects, and the Character methods that inspect single letters.



IN THIS CHAPTER

- Wrapper classes
- parseInt and friends
- Autoboxing
- Unboxing
- Character methods
- ASCII

Wrapper Classes

A primitive type is only a value. It has no methods. To use values where an **object** is required, and to get helpful utility methods, Java gives every primitive a matching class that **wraps** it. All of them are in `java.lang`, so no import is needed.

PRIMITIVE	WRAPPER CLASS	CONVERT A STRING WITH
byte	Byte	Byte.parseByte(s)
short	Short	Short.parseShort(s)
int	**Integer**	Integer.parseInt(s)
long	Long	Long.parseLong(s)
float	Float	Float.parseFloat(s)
double	Double	Double.parseDouble(s)
char	**Character**	(no parse; use charAt)
boolean	Boolean	Boolean.parseBoolean(s)



KEY POINT

Only **int** and **char** have wrapper names that differ from the primitive: **Integer** and **Character**. Others just start with a capital letter.

The primitive types are the simple built-in types. A wrapper is a **composite (class) type**: its variable holds a reference to an object, can be `null`, and can call methods. Through classes, Java also lets you invent your own types, such as `Student`.

Converting Text to Numbers

Anything you type or pass in `args` arrives as a **String**. The parse methods are **static** methods of the wrapper classes that turn the text into a number.

METHOD	RETURNS	EXAMPLE	RESULT
int Integer.parseInt(String s)	int	Integer.parseInt("250")	250
long Long.parseLong(String s)	long	Long.parseLong("9876543210")	9876543210
float Float.parseFloat(String s)	float	Float.parseFloat("2.5")	2.5
double Double.parseDouble(String s)	double	Double.parseDouble("7.25")	7.25

```
public class ParseDemo
{
    public static void main(String[] args)
    {
        String a = "250", b = "17";
        System.out.println(a + b);
        System.out.println(Integer.parseInt(a) + Integer.parseInt(b));
        double d = Double.parseDouble("7.25") * 2;
        System.out.println(d);
        long big = Long.parseLong("9876543210");
        System.out.println(big + 1);
        float f = Float.parseFloat("2.5");
        System.out.println(f / 2);
        // Integer.parseInt("12x") would throw NumberFormatException
    }
}
```

```
Output
------------------------------
25017
267
14.5
9876543211
1.25
```



BOARD ALERT

If the text is not a valid number, for example "12x" or "", the program stops with a **NumberFormatException** at run time. Spaces inside the string also make it invalid.

The opposite direction is also common: `String.valueOf(25)` or `25 + ""` turns a number into text.

Autoboxing and Unboxing

TERM	DEFINITION	EXAMPLE
Autoboxing	Automatic conversion of a **primitive** into an object of its wrapper class	`Integer x = 50;`
Unboxing	Automatic conversion of a **wrapper object** back into its primitive	`int y = x;`

```
public class BoxUnbox
{
    public static void main(String[] args)
    {
        int p = 40;
        Integer boxed = p;            // autoboxing
        Double dd = 3.5;              // autoboxing
        char c = 'Z';
        Character ch = c;             // autoboxing
        int q = boxed;                // unboxing
        double sum = boxed + dd;      // both unboxed, then added
        System.out.println(boxed + " " + dd + " " + ch);
        System.out.println(q + 1);
        System.out.println(sum);
    }
}
```

```
Output
------------------------------
40 3.5 Z
41
43.5
```



SIDEKICK TIP

Memory trick: **box** = put the value into a box (object). **Unbox** = take it out of the box. Java does both silently when the types call for it.

The Character Class

All of these are **static** methods, so you call them on the class: `Character.isDigit('7')`. The argument is a single `char`.

METHOD	RETURNS	TRUE / RESULT WHEN
isDigit(char ch)	boolean	ch is a digit 0 to 9
isLetter(char ch)	boolean	ch is an alphabet letter
isLetterOrDigit(char ch)	boolean	ch is a letter or a digit
isLowerCase(char ch)	boolean	ch is a small letter
isUpperCase(char ch)	boolean	ch is a capital letter
isWhitespace(char ch)	boolean	ch is a space, tab or newline
toLowerCase(char ch)	char	the lower case form (others unchanged)
toUpperCase(char ch)	char	the upper case form (others unchanged)

```
public class CharLab
{
    public static void main(String[] args)
    {
        System.out.println(Character.isDigit('7'));
        System.out.println(Character.isLetter('#'));
        System.out.println(Character.isLetterOrDigit('_'));
        System.out.println(Character.isLowerCase('g'));
        System.out.println(Character.isUpperCase('g'));
        System.out.println(Character.isWhitespace('\t'));
        System.out.println(Character.toUpperCase('q'));
        System.out.println(Character.toLowerCase('Q'));
        System.out.println(Character.toUpperCase('5'));
        System.out.println(Character.toUpperCase('a') + 1);
        System.out.println((char) (Character.toUpperCase('a') + 1));
    }
}
```

```
Output
------------------------------
true
false
false
true
false
true
Q
q
5
66
B
```



BOARD ALERT

`Character.toUpperCase('a') + 1` prints **66**, because a `char` plus an `int` is an `int` (the ASCII code 65 + 1). Cast with `(char)` to get B.

Classifying the Characters of a Sentence

```
import java.util.*;
public class CharCount
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int up = 0, lo = 0, dg = 0, sp = 0, ot = 0;
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (Character.isUpperCase(c))      up++;
            else if (Character.isLowerCase(c)) lo++;
            else if (Character.isDigit(c))     dg++;
            else if (Character.isWhitespace(c)) sp++;
            else                               ot++;
        }
        System.out.println("Upper=" + up + " Lower=" + lo + " Digits=" + dg);
        System.out.println("Spaces=" + sp + " Others=" + ot);
    }
}
```

```
Input: Room 42, Block-B!
----------------------------
Upper=3 Lower=7 Digits=2
Spaces=2 Others=3
```

CHARACTERS	CATEGORY	COUNT
R, B, B	upper	3
o, o, m, l, o, c, k	lower	7
4, 2	digit	2
two spaces	whitespace	2
, - !	others	3

Characters and ASCII

Every character has a numeric code. In ASCII: **'0' to '9' = 48 to 57**, **'A' to 'Z' = 65 to 90** and **'a' to 'z' = 97 to 122**. A capital letter and its small letter differ by exactly **32**. This lets you compute with characters.

```
public class AsciiPlay
{
    public static void main(String[] args)
    {
        char c = 'D';
        int code = c;                    // widening: char to int
        System.out.println(code);
        char next = (char) (c + 3);      // shift forward by 3
        System.out.println(next);
        char small = (char) (c + 32);
        System.out.println(small);
        char dig = '7';
        int value = dig - '0';           // digit character to number
        System.out.println(value * 2);
        System.out.println('A' + 'B');
        System.out.println("" + 'A' + 'B');
    }
}
```

```
Output
------------------------------
68
G
d
14
131
AB
```



SIDEKICK TIP

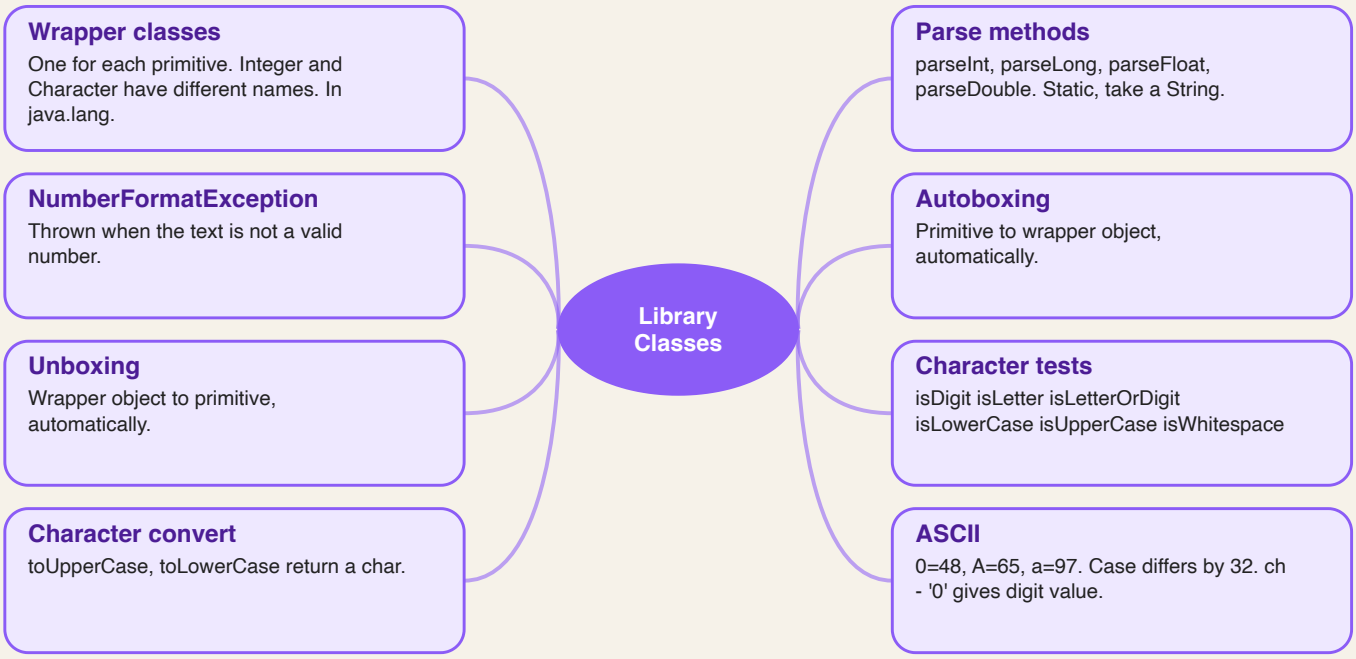
To convert a digit character to its value, use `ch - '0'`. To convert a number 0 to 9 to a character, use `(char)(n + '0')`.



REAL LIFE

Password rules (at least one capital, one digit, one symbol) are checked with the same Character methods you just met, character by character.

Mind Map Recap



HOT Questions

HOT QUESTION

What is the output of `System.out.println(Character.isLetter('9') || Character.isDigit('9'));`? (Answer: true. The first part is false, the second true.)

HOT QUESTION

Differentiate between `Integer.parseInt("5")` and `Integer.valueOf("5")`. (Answer: parseInt gives the primitive int 5; valueOf gives an Integer object, which is auto-unboxed when needed.)

HOT QUESTION

Is the statement `int n = Integer.parseInt("3.5");` valid? (Answer: it compiles but throws NumberFormatException at run time, because "3.5" is not an int.)

06

CHAPTER · COMPUTER APPLICATIONS

Encapsulation

Good classes keep their data safe. Encapsulation hides the data behind access rules and controls where each variable can be seen and how long it lives.



IN THIS CHAPTER

- Access specifiers
- Visibility rules
- Getters and setters
- Scope of variables
- Four kinds of variables

What Is Encapsulation?

Encapsulation means binding the data (variables) and the code that works on it (methods) into one unit, the class, and **restricting direct access** to the data. Outsiders must use the public methods you provide. It is data hiding with a controlled doorway.



REAL LIFE

An ATM never lets you reach into the bank's database. You can only press options (deposit, withdraw, balance) and the machine checks the rules each time. The account data is encapsulated.

Access Specifiers

An access specifier (access modifier) decides **who may see a member** of a class, whether a variable or a method.

SPECIFIER	SAME CLASS	SAME PACKAGE	SUBCLASS IN ANOTHER PACKAGE	ANYWHERE ELSE
private	Yes	No	No	No
(none: default)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

- **private:** the tightest. Visible only inside its own class. Used for data.
- **protected:** visible in its class, the same package and in subclasses (even in other packages).
- **public:** visible everywhere. Used for the methods you want to share.
- **default** (no keyword): visible within the same package only.



KEY POINT

Visibility order, narrowest to widest: **private < default < protected < public**.

```
class Account
{
    private double balance = 500;     // hidden data
    public double getBalance()        // public doorway
    {
        return balance;
    }
}

// in another class:
Account a = new Account();
System.out.println(a.balance);        // ERROR: balance has private access
System.out.println(a.getBalance());   // fine
```

Getters and Setters: The Controlled Doorway

Keep variables `private` and offer a **getter** (reads the value) and a **setter** (changes it after checking). The setter can reject bad data.

```
class Account
{
    private int accNo;
    private double balance;

    Account(int accNo, double opening)
    {
        this.accNo = accNo;
        balance = opening;
    }
    public double getBalance()
    {
        return balance;
    }
    public void deposit(double amt)
    {
        if (amt > 0)
            balance += amt;
        else
            System.out.println("Invalid amount");
    }
    public void withdraw(double amt)
    {
        if (amt > 0 && amt ⩽ balance)
            balance -= amt;
        else
            System.out.println("Cannot withdraw " + amt);
    }
}
public class BankDemo
{
    public static void main(String[] args)
    {
        Account a = new Account(101, 500);
        a.deposit(250);
        a.withdraw(2000);
        a.deposit(-5);
        a.withdraw(300);
        System.out.println("Balance: " + a.getBalance());
    }
}
```

```
Output
------------------------------
Cannot withdraw 2000.0
Invalid amount
Balance: 450.0
```

SIDEKICK TIP

If `balance` were public, anyone could write `a.balance = -99999;`. Making it private and checking inside the methods is the whole point of encapsulation.

Scope and Lifetime of Variables

The **scope** of a variable is the region of the program where its name can be used. Its **lifetime** is how long it stays in memory. Java has four kinds of variables.

KIND	DECLARED	SCOPE	LIFETIME	DEFAULT VALUE
Local	Inside a method or block	Only that method or block	From declaration until the block ends	None: must be assigned before use
Argument (parameter)	In the method header	Whole method body	During the method call	Receives the actual argument
Instance	Inside the class, outside methods, no `static`	All non-static methods of the class	As long as the object lives	0, 0.0, false, null
Class (static)	Inside the class with `static`	Whole class; shared by all objects	Whole program run	0, 0.0, false, null

An **instance variable** gets a separate copy for every object. A **class variable** has just **one copy shared** by all objects, which makes it perfect for counters.

```
public class ScopeDemo
{
    static int count = 0;          // class variable (one shared copy)
    int id;                        // instance variable (one per object)

    ScopeDemo()
    {
        count++;
        id = count;
    }
    int addTo(int n)               // n is an argument variable
    {
        int total = id + n;        // total is a local variable
        return total;
    }
    public static void main(String[] args)
    {
        ScopeDemo a = new ScopeDemo();
        ScopeDemo b = new ScopeDemo();
        ScopeDemo c = new ScopeDemo();
        System.out.println(a.id + " " + b.id + " " + c.id);
        System.out.println(ScopeDemo.count);
        System.out.println(c.addTo(10));
        for (int i = 0; i < 2; i++)
        {
            int sq = i * i;                 // local to the loop block
            System.out.print(sq + " ");
        }
        System.out.println();
        // System.out.println(sq);   // ERROR: sq is out of scope here
    }
}
```

```
Output
----------------------------
1 2 3
3
13
0 1
```

VARIABLE	KIND	WHY
count	class (static)	Declared with static; shared, ends at 3
id	instance	Each object holds its own id
n	argument	Listed in the header of addTo
total	local	Declared inside addTo
i, sq	local (block)	Exist only inside the for loop

Name Clashes and Shadowing

A local or argument variable with the same name as an instance variable **hides** it inside that method. Use `this.name` to reach the instance variable.

```
public class Shadow
{
    int x = 10;

    void test(int x)
    {
        System.out.println(x);          // the argument
        System.out.println(this.x);     // the instance variable
        x = x + 1;
        this.x = this.x + 100;
    }
    public static void main(String[] args)
    {
        Shadow s = new Shadow();
        s.test(5);
        System.out.println(s.x);
    }
}
```

```
Output
----------------------------
5
10
110
```



BOARD ALERT

A local variable must be **initialised before use**. `int t; System.out.println(t);` gives the compile error *variable t might not have been initialized*. Instance variables do not have this problem, they get default values.

Identify the Variables

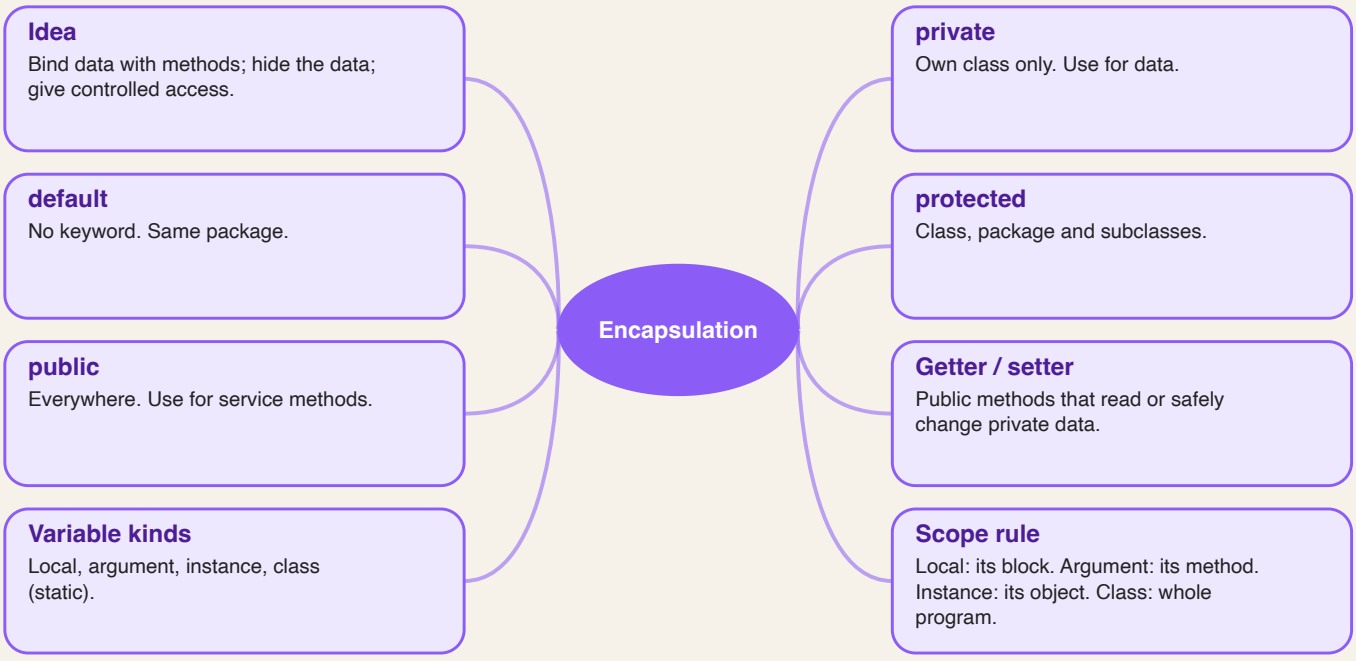
A favourite board and lab question: label each variable in a code snippet.

```
public class Shop
{
    static int shops;          // (a)
    private String owner;      // (b)
    public double rate;        // (c)

    public double bill(int qty)      // (d) qty
    {
        double amt = qty * rate;     // (e) amt
        return amt;
    }
}
```

LABEL	VARIABLE	KIND	ACCESS
(a)	shops	Class (static) variable	default (package)
(b)	owner	Instance variable	private
(c)	rate	Instance variable	public
(d)	qty	Argument (formal parameter)	-
(e)	amt	Local variable	-

Mind Map Recap



HOT Questions

HOT QUESTION

A static variable `int c;` is incremented in the constructor. After creating 4 objects, what do all four objects see for c? (Answer: 4, because there is one shared copy.)

HOT QUESTION

Name the access specifier that would make a method visible to every class in every package. (Answer: public.)

HOT QUESTION

Why is it considered a bad idea to declare every instance variable `public`? (Answer: outside code could set invalid values and bypass checks. Private data plus public methods protects the object's consistency.)

07

CHAPTER · COMPUTER APPLICATIONS

Arrays

One variable holds one value. An array holds a whole row of them under a single name. Master arrays and you can sort a class list, search a record and fill a matrix.



IN THIS CHAPTER

- 1D arrays
- Input and display
- Selection sort
- Bubble sort
- Linear search
- Binary search
- 2D arrays
- Diagonals

What Is an Array?

An **array** is a fixed-size collection of elements of the **same data type**, stored in neighbouring memory cells and reached by a number called the **index** or **subscript**. Indexes start at **0**. An array is a **composite (reference) type**: the variable holds the address of the array object.

- **Single dimensional array (1D):** a simple list.
- **Double dimensional array (2D):** a table of rows and columns (a matrix).
- **Length:** `arr.length` is a **field** (no brackets). It is different from `s.length()` of a String, which is a method.

Declaring, Creating and Initialising

```
int[] marks;                    // declare (also legal: int marks[];)
marks = new int[5];             // create: five ints, all 0
double[] price = new double[3]; // declare and create together
int[] a = {12, 7, 30, 4, 19};   // create and fill in one step
String[] days = {"Mon", "Tue", "Wed"};
```

INDEX	0	1	2	3	4
a[ ]	12	7	30	4	19



KEY POINT

The last valid index is `length − 1`. Using `a[5]` for a 5-element array throws **ArrayIndexOutOfBoundsException** at run time (the compiler does not catch it). A new array is filled with default values: 0, 0.0, false, '\u0000' or null.

Input, Traversal and Simple Calculations

```
import java.util.*;
public class ArrayBasics
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] a = new int[5];
        for (int i = 0; i < a.length; i++)       // accept
            a[i] = sc.nextInt();
        int sum = 0, max = a[0], min = a[0];
        for (int i = 0; i < a.length; i++)       // process
        {
            sum += a[i];
            if (a[i] > max) max = a[i];
            if (a[i] < min) min = a[i];
        }
        System.out.print("Reversed: ");
        for (int i = a.length - 1; i ⩾ 0; i--)  // display backwards
            System.out.print(a[i] + " ");
        System.out.println();
        System.out.println("Sum=" + sum + " Max=" + max + " Min=" + min);
        System.out.println("Average=" + (double) sum / a.length);
    }
}
```

```
Input: 12 7 30 4 19
-----------------------------
Reversed: 19 4 30 7 12
Sum=72 Max=30 Min=4
Average=14.4
```



SIDEKICK TIP

When averaging, cast one operand: `(double) sum / n`. Plain `sum / n` divides two ints and loses the decimal part.

Copying an Array: Beware the Alias

```
public class CopyArr
{
    public static void main(String[] args)
    {
        int[] a = {1, 2, 3};
        int[] b = a;                    // alias: same array, two names
        b[0] = 99;
        System.out.println(a[0]);
        int[] c = new int[a.length];    // real copy
        for (int i = 0; i < a.length; i++)
            c[i] = a[i];
        c[1] = -5;
        System.out.println(a[1] + " " + c[1]);
    }
}
```

```
Output
----------------------------
99
2 -5
```

Sorting: Selection Sort

Sorting arranges the elements in ascending or descending order. **Selection sort** repeatedly **selects** the smallest remaining element and puts it at the front of the unsorted part.

1. Pass 1
Find the smallest in positions 0 to n-1. Swap it with position 0.

2. Pass 2
Find the smallest in positions 1 to n-1. Swap with position 1.

3. Continue
After n-1 passes the array is sorted. Position `i` is final after pass `i+1`.

```
public class SelSort
{
    public static void main(String[] args)
    {
        int[] a = {29, 10, 14, 37, 8};
        int n = a.length;
        for (int i = 0; i < n - 1; i++)
        {
            int pos = i;                    // position of smallest so far
            for (int j = i + 1; j < n; j++)
                if (a[j] < a[pos])
                    pos = j;
            int t = a[i];  a[i] = a[pos];  a[pos] = t;
            System.out.print("Pass " + (i + 1) + ": ");
            for (int k = 0; k < n; k++)
                System.out.print(a[k] + " ");
            System.out.println();
        }
    }
}
```

```
Output
----------------------------
Pass 1: 8 10 14 37 29
Pass 2: 8 10 14 37 29
Pass 3: 8 10 14 37 29
Pass 4: 8 10 14 29 37
```



REAL LIFE

Selection sort is how you might sort cards in your hand: pick the lowest card, place it first, then pick the lowest of the rest.

Sorting: Bubble Sort

Bubble sort compares **neighbouring** pairs and swaps them if they are out of order. After each pass the largest remaining element has bubbled to the end.

```
public class BubSort
{
    public static void main(String[] args)
    {
        int[] a = {29, 10, 14, 37, 8};
        int n = a.length;
        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - 1 - i; j++)
            {
                if (a[j] > a[j + 1])
                {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
            }
            System.out.print("Pass " + (i + 1) + ": ");
            for (int k = 0; k < n; k++)
                System.out.print(a[k] + " ");
            System.out.println();
        }
    }
}
```

```
Output
------------------------------
Pass 1: 10 14 29 8 37
Pass 2: 10 14 8 29 37
Pass 3: 10 8 14 29 37
Pass 4: 8 10 14 29 37
```

	SELECTION SORT	BUBBLE SORT
Idea	Select the minimum, place it in position	Swap adjacent out-of-order pairs
Swaps per pass	At most one	Possibly many
Inner loop runs	`j = i+1` to n−1	`j = 0` to n−2−i
Largest ends at the back	Not necessarily after each pass	Yes, after each pass
Passes needed	n - 1	n - 1



SIDEKICK TIP

For **descending** order, flip only the comparison sign: `a[j] < a[pos]` becomes `a[j] > a[pos]` (selection), and `a[j] > a[j+1]` becomes `a[j] < a[j+1]` (bubble).

Searching: Linear Search

Check the elements one by one from the start until the key is found or the array ends. It works on **any** array, sorted or not.

```
import java.util.*;
public class LinSearch
{
    public static void main(String[] args)
    {
        int[] a = {16, 3, 42, 8, 25, 11};
        Scanner sc = new Scanner(System.in);
        System.out.print("Search for: ");
        int key = sc.nextInt();
        int pos = -1;
        for (int i = 0; i < a.length; i++)
        {
            if (a[i] == key)
            {
                pos = i;
                break;
            }
        }
        if (pos == -1)
            System.out.println(key + " not found");
        else
            System.out.println(key + " found at index " + pos);
    }
}
```

```
Input: 8
----------------------------
Search for: 8 found at index 3
```

Searching: Binary Search

Works **only on a sorted array**. Look at the middle element: if it equals the key, done; if the key is smaller, search the left half; otherwise search the right half. Each step halves the work.

```
public class BinSearch
{
    public static void main(String[] args)
    {
        int[] a = {5, 12, 18, 23, 31, 47, 60};
        int key = 47;
        int low = 0, high = a.length - 1, pos = -1;
        while (low <= high)
        {
            int mid = (low + high) / 2;
            System.out.println("low=" + low + " high=" + high + " mid=" + mid);
            if (a[mid] == key)
            {
                pos = mid;
                break;
            }
            else if (key < a[mid])
                high = mid - 1;
            else
                low = mid + 1;
        }
        System.out.println(pos == -1 ? "Not found" : "Found at " + pos);
    }
}
```

```
Output
-----------------------------
low=0 high=6 mid=3
low=4 high=6 mid=5
Found at 5
```

STEP	LOW	HIGH	MID	A[MID]	DECISION
1	0	6	3	23	47 > 23, search right: low = 4
2	4	6	5	47	Equal. Found at index 5

POINT	LINEAR SEARCH	BINARY SEARCH
Array must be sorted?	No	**Yes**
Method	Compare one by one	Halve the range each time
Speed for large data	Slow	Very fast
Worst case for 1000 items	1000 comparisons	about 10 comparisons



BOARD ALERT

Using binary search on an unsorted array can miss elements that are present. Always sort first, or use linear search.

Double Dimensional Arrays

A 2D array is an array of arrays: a table with **rows** and **columns**. An element needs two subscripts, `m[row][col]`, and both start at 0.

```
int[][] m = new int[3][4];            // 3 rows, 4 columns
int[][] n = { {1, 2, 3},
              {4, 5, 6},
              {7, 8, 9} };             // 3 x 3 filled directly
// n.length = number of rows = 3
// n[0].length = columns in row 0 = 3
```

	COL 0	COL 1	COL 2
row 0	1	2	3
row 1	4	5	6
row 2	7	8	9

```
import java.util.*;
public class Matrix
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[][] m = new int[3][3];
        for (int i = 0; i < 3; i++)          // accept row by row
            for (int j = 0; j < 3; j++)
                m[i][j] = sc.nextInt();
        for (int i = 0; i < 3; i++)          // display in matrix form
        {
            for (int j = 0; j < 3; j++)
                System.out.print(m[i][j] + "\t");
            System.out.println();
        }
    }
}
```

```
Input: 1 2 3 | 4 5 6 | 7 8 9
-----------------------------
1       2       3
4       5       6
7       8       9
```

Row Sums, Column Sums and Diagonals

In a square matrix of order n: the **main (left) diagonal** has **i == j**; the **right (secondary) diagonal** has **i + j == n - 1**.

```
public class MatrixSums
{
    public static void main(String[] args)
    {
        int[][] m = { {2, 4, 6}, {1, 3, 5}, {7, 8, 9} };
        int n = 3, left = 0, right = 0;
        for (int i = 0; i < n; i++)
        {
            int rs = 0, cs = 0;
            for (int j = 0; j < n; j++)
            {
                rs += m[i][j];          // row i
                cs += m[j][i];          // column i
            }
            System.out.println("Row " + i + " = " + rs
                               + ", Col " + i + " = " + cs);
            left += m[i][i];
            right += m[i][n - 1 - i];
        }
        System.out.println("Left diagonal = " + left);
        System.out.println("Right diagonal = " + right);
    }
}
```

```
Output
------------------------------
Row 0 = 12, Col 0 = 10
Row 1 = 9, Col 1 = 15
Row 2 = 24, Col 2 = 20
Left diagonal = 14
Right diagonal = 16
```

	C0	C1	C2	ROW SUM
r0	**2**	4	**6**	12
r1	1	**3**	5	9
r2	**7**	8	**9**	24
Column sum	10	15	20	

Left diagonal cells (i = j): 2, 3, 9 = 14. Right diagonal cells (i + j = 2): 6, 3, 7 = 16. (Bold cells above mark both diagonals.)



SIDEKICK TIP

Transposing swaps rows and columns: the element at `[i][j]` goes to `[j][i]`. The same inner-loop trick (`m[j][i]`) gave us column sums above.

Passing and Returning Arrays

```
public class ArrayMethods
{
    static int largest(int[] x)
    {
        int m = x[0];
        for (int v : x)                  // enhanced for loop
            if (v > m) m = v;
        return m;
    }
    static void doubleAll(int[] x)
    {
        for (int i = 0; i < x.length; i++)
            x[i] *= 2;
    }
    public static void main(String[] args)
    {
        int[] d = {4, 9, 2};
        System.out.println(largest(d));
        doubleAll(d);
        System.out.println(d[0] + " " + d[1] + " " + d[2]);
    }
}
```

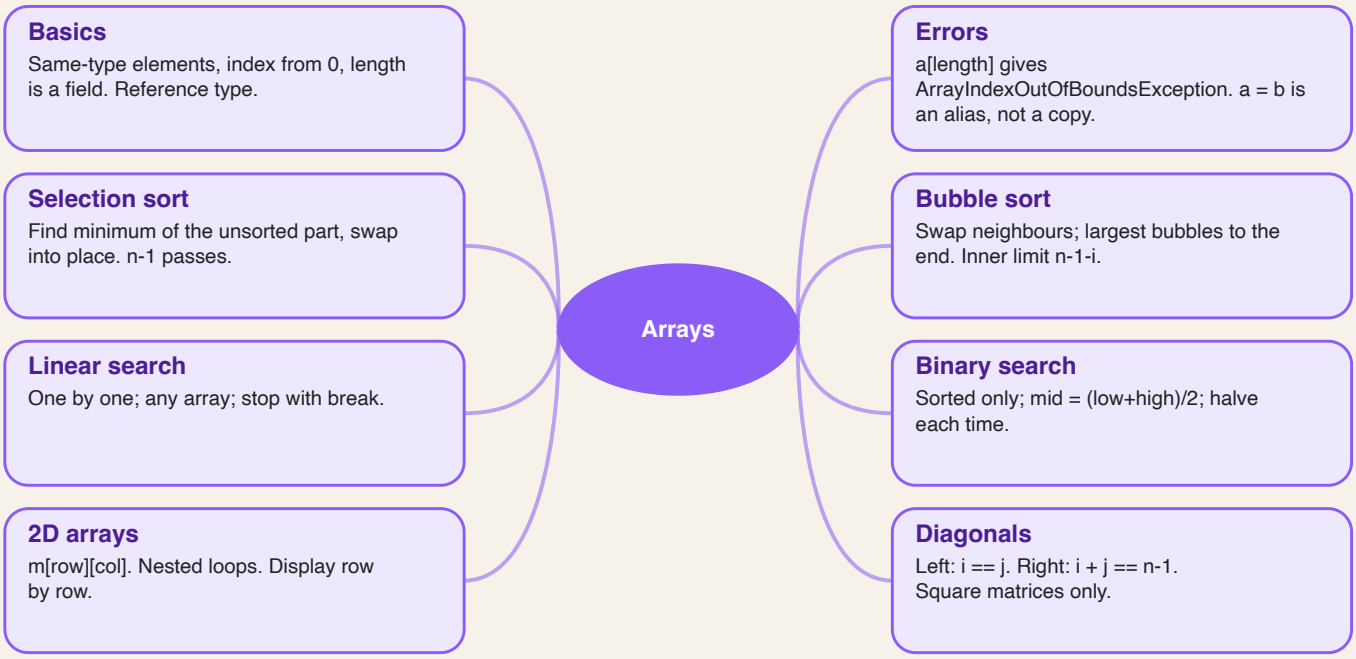
```
Output
------------------------------
9
8 18 4
```



FUN FACT

The enhanced `for (int v : x)` reads every element without an index. It is handy, but you cannot change the array through v, and board answers normally use the classic indexed loop.

Mind Map Recap



HOT Questions

HOT QUESTION

What will be the output? `int[] a = {3, 1, 2}; int[] b = a; b[1] = 9; System.out.println(a[1]);` (Answer: 9, because b is only another name for the same array.)

HOT QUESTION

A sorted array has 1000 elements. Roughly how many comparisons does binary search need at most? (Answer: about 10, since $2^{10} = 1024$.)

HOT QUESTION

For a 4 × 4 matrix, which cells lie on both diagonals? (Answer: none, since i = j and i + j = 3 cannot both hold for whole numbers. For odd orders such as 3 × 3, the centre cell is shared.)

08

CHAPTER · COMPUTER APPLICATIONS

String Handling

Names, messages, passwords, sentences: most real data is text. The String class gives you a rich set of methods to slice, compare, search and change it.



IN THIS CHAPTER

- String basics
- All 18 methods
- Output tables
- Extracting characters
- Palindrome, pig Latin
- String arrays
- Sorting names

What Is a String?

A **String** is a sequence of characters written in double quotes. It is a **class**, not a primitive type, so a String variable is a reference to a String object. Strings in Java are **immutable**: once created they never change. Every method that seems to modify a string actually returns a **new** string.

```
String a = "Java";
String b = new String("Java");
String c = "";                 // empty string, length 0
char   d = 'J';                // a char is NOT a String
```



BOARD ALERT

Because strings are immutable, `s.toUpperCase();` on its own does nothing to `s`. You must capture the result: `s = s.toUpperCase();`.

Characters are numbered from **0**. The last index of a string is `length() − 1`.

INDEX	0	1	2	3	4	5	6
Char	S	I	D	E	K	I	T

The String Methods in the Syllabus

METHOD	RETURNS	WHAT IT DOES
int length()	int	Number of characters, including spaces
char charAt(int n)	char	Character at index n (error if n is out of range)
int indexOf(char ch)	int	Index of the **first** occurrence of ch, or -1
int lastIndexOf(char ch)	int	Index of the **last** occurrence of ch, or -1
String trim()	String	Removes spaces at the start and the end only
String toLowerCase()	String	All letters to small
String toUpperCase()	String	All letters to capital
String concat(String str)	String	Joins str after this string (same as +)
String replace(char old, char new)	String	Replaces every old character by the new one
String substring(int b)	String	From index b to the end
String substring(int b, int e)	String	From b up to **e - 1** (e is excluded)
boolean equals(String str)	boolean	True if identical, case matters
boolean equalsIgnoreCase(String str)	boolean	True if identical ignoring case
int compareTo(String str)	int	Dictionary comparison; case matters
int compareToIgnoreCase(String str)	int	Dictionary comparison ignoring case
boolean startsWith(String str)	boolean	True if it begins with str
boolean endsWith(String str)	boolean	True if it ends with str
String String.valueOf(any type)	String	Converts int, double, char, boolean etc. to text

Outputs of Every Method

```
public class StrLab1
{
    public static void main(String[] args)
    {
        String s = "  Sidekick Java  ";
        System.out.println("[" + s.trim() + "]");
        System.out.println(s.length());
        System.out.println(s.trim().length());
        String t = "Programming";
        System.out.println(t.charAt(0) + " " + t.charAt(10));
        System.out.println(t.indexOf('g'));
        System.out.println(t.lastIndexOf('g'));
        System.out.println(t.indexOf('z'));
        System.out.println(t.toUpperCase());
        System.out.println(t.toLowerCase());
        System.out.println(t.substring(3));
        System.out.println(t.substring(3, 7));
        System.out.println(t.replace('m', 'M'));
        System.out.println(t.concat(" is fun"));
        System.out.println(t);
    }
}
```

```
Output
------------------------------
[Sidekick Java]
17
13
P g
3
10
-1
PROGRAMMING
programming
gramming
gram
PrograMMing
Programming is fun
Programming
```



KEY POINT

`substring(3, 7)` returns characters at indexes **3, 4, 5, 6**, that is 4 characters (7 − 3). The end index is **excluded**. The last line shows `t` is unchanged after all those calls.

```
public class StrLab2
{
    public static void main(String[] args)
    {
        String a = "Apple", b = "apple", c = "Banana";
        System.out.println(a.equals(b));
        System.out.println(a.equalsIgnoreCase(b));
        System.out.println(a.compareTo(b));
        System.out.println(a.compareToIgnoreCase(b));
        System.out.println(a.compareTo(c));
        System.out.println(c.compareTo(a));
        System.out.println("Cat".compareTo("Cattle"));
        System.out.println("Hello".startsWith("He"));
        System.out.println("Hello".endsWith("lo"));
        System.out.println("Hello".endsWith("LO"));
        System.out.println(String.valueOf(45) + 5);
        System.out.println(String.valueOf(3.5) + String.valueOf(true));
        System.out.println(String.valueOf('Q').length());
    }
}
```

```
Output
------------------------------
false
true
-32
0
-1
1
-3
true
true
false
455
3.5true
1
```

How compareTo Works

`compareTo` compares two strings character by character using Unicode values.

1. Find the first position where the characters differ

The result is `this.charAt(i) – other.charAt(i)`. In `"Apple".compareTo("apple")`: 'A' (65) - 'a' (97) = **-32**

.

2. If one string is a prefix of the other

The result is the difference of the **lengths**

. `"Cat".compareTo("Cattle")` = 3 - 6 = **-3**

.

3. If both are identical

The result is **0**

.

RESULT OF A.COMPARETO(B)	MEANING
Negative	a comes **before** b in dictionary order
Zero	Both strings are equal
Positive	a comes **after** b



BOARD ALERT

== vs equals. == checks whether two names point to the *same object*; `equals()` checks whether the *text* is the same. To compare text, always use `equals`.



SIDEKICK TIP

Capital letters (65 to 90) sort **before** small letters (97 to 122) in compareTo. Use `compareToIgnoreCase` when sorting names that may differ in case.

Joining with + and Mixed Types

```
public class PlusTrap
{
    public static void main(String[] args)
    {
        System.out.println(1 + 2 + "A");
        System.out.println("A" + 1 + 2);
        System.out.println('a' + 'b' + "c");
        System.out.println("c" + 'a' + 'b');
        System.out.println("" + 4 * 5 + 1);
    }
}
```

```
Output
----------------------------
3A
A12
195c
cab
201
```

EXPRESSION	EVALUATION (LEFT TO RIGHT)	RESULT
1 + 2 + "A"	3 first (both ints), then "3" + "A"	3A
"A" + 1 + 2	"A1" then "A12"	A12
'a' + 'b' + "c"	97 + 98 = 195, then "195c"	195c
"c" + 'a' + 'b'	string joins each char	cab
"" + 4 * 5 + 1	"" + 20, then + 1	201

Extracting and Counting Characters

Most string programs follow one loop: **go through every index, take `charAt(i)`, decide, act.**

```
import java.util.*;
public class VowelCount
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        int v = 0, c = 0, w = 1;
        for (int i = 0; i < s.length(); i++)
        {
            char ch = s.charAt(i);
            if ("aeiou".indexOf(ch) != -1)
                v++;
            else if (Character.isLetter(ch))
                c++;
            else if (ch == ' ')
                w++;
        }
        System.out.println("Vowels=" + v + " Consonants=" + c);
        System.out.println("Words=" + w);
    }
}
```

```
Input: Computer Applications is fun
----------------------------
Vowels=10 Consonants=15
Words=4
```



SIDEKICK TIP

The `"aeiou".indexOf(ch) != -1` trick checks 'is ch one of these letters?' without five comparisons.

Reversing and Palindrome

A **palindrome** reads the same forwards and backwards (MADAM, LEVEL, NITIN). Build the reverse by adding each character in front of the previous result, then compare with `equals`.

```
import java.util.*;
public class Palin
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        String rev = "";
        for (int i = s.length() - 1; i ⩾ 0; i--)
            rev = rev + s.charAt(i);
        System.out.println("Reverse: " + rev);
        if (s.equalsIgnoreCase(rev))
            System.out.println(s + " is a palindrome");
        else
            System.out.println(s + " is not a palindrome");
    }
}
```

```
Input: Level
-----------------------------
Reverse: leveL
Level is a palindrome
```

I	S.CHARAT(I)	REV AFTER THE STEP
4	l	l
3	e	le
2	v	lev
1	e	leve
0	L	leveL

`equalsIgnoreCase` then says `Level` and `leveL` match, so it is a palindrome.

Pig Latin

Rule for this book: if a word starts with a vowel, add **"ay"** to the end. Otherwise move every letter before the first vowel to the end and then add **"ay"**. Example: *trouble* → *oubletray*, *apple* → *appleay*.

```
import java.util.*;
public class PigLatin
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String w = sc.next().toLowerCase();
        int k = -1;
        for (int i = 0; i < w.length(); i++)
        {
            if ("aeiou".indexOf(w.charAt(i)) != -1)
            {
                k = i;                    // index of the first vowel
                break;
            }
        }
        String res;
        if (k == -1)
            res = w + "ay";               // no vowel at all
        else
            res = w.substring(k) + w.substring(0, k) + "ay";
        System.out.println(res);
    }
}
```

```
Input: trouble
----------------------------
oubletray
```

PART	EXPRESSION	VALUE
First vowel index	k	2 (the letter o)
From the vowel on	w.substring(2)	ouble
Letters before it	w.substring(0, 2)	tr
Result	ouble + tr + ay	oubletray

Working Word by Word

To handle a sentence, locate spaces with `indexOf(' ')` and cut words with `substring`. A trailing space on the sentence makes the last word easy to catch, so add one first.

```
import java.util.*;
public class WordSplit
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().trim() + " ";
        String longest = "";
        int count = 0;
        while (s.length() > 0)
        {
            int sp = s.indexOf(' ');
            String word = s.substring(0, sp);
            count++;
            if (word.length() > longest.length())
                longest = word;
            s = s.substring(sp + 1);
        }
        System.out.println("Words: " + count);
        System.out.println("Longest: " + longest);
    }
}
```

```
Input:   Practice makes programming perfect
------------------------------
Words: 4
Longest: programming
```



BOARD ALERT

This version expects **single spaces** between words. Two spaces in a row would give an empty word. In the exam the question will usually state the words are separated by a single space.

Changing Case Letter by Letter

```
public class CaseSwap
{
    public static void main(String[] args)
    {
        String s = "Hello World 2027";
        String out = "";
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (Character.isUpperCase(c))
                out += Character.toLowerCase(c);
            else if (Character.isLowerCase(c))
                out += Character.toUpperCase(c);
            else
                out += c;
        }
        System.out.println(out);
    }
}
```

```
Output
------------------------------
hELLO wORLD 2027
```

String Arrays

An array can hold Strings too. `names.length` counts the elements; `names[i].length()` counts the characters of one element.

```
public class StrArray
{
    public static void main(String[] args)
    {
        String[] city = {"Pune", "Delhi", "Agra", "Mumbai", "Goa"};
        System.out.println(city.length + " cities");
        System.out.println(city[1].length());
        System.out.println(city[3].charAt(0));
        String key = "Agra";
        int pos = -1;
        for (int i = 0; i < city.length; i++)       // linear search
        {
            if (city[i].equals(key))
            {
                pos = i;
                break;
            }
        }
        System.out.println(key + " at " + pos);
    }
}
```

```
Output
-----------------------------
5 cities
5
M
Agra at 2
```

Alphabetical Order: Sorting a String Array

Sort with the same two algorithms you met for numbers, replacing the comparison `a[j] > a[j+1]` by `a[j].compareTo(a[j+1]) > 0`.

```
public class SortNames
{
    public static void main(String[] args)
    {
        String[] n = {"Meera", "arjun", "Zoya", "Kabir", "Isha"};
        int len = n.length;
        for (int i = 0; i < len - 1; i++)           // bubble sort
        {
            for (int j = 0; j < len - 1 - i; j++)
            {
                if (n[j].compareToIgnoreCase(n[j + 1]) > 0)
                {
                    String t = n[j];
                    n[j] = n[j + 1];
                    n[j + 1] = t;
                }
            }
        }
        for (int i = 0; i < len; i++)
            System.out.println(n[i]);
    }
}
```

```
Output
-----------------------------
arjun
Isha
Kabir
Meera
Zoya
```

```
public class SelNames
{
    public static void main(String[] args)
    {
        String[] n = {"Rohan", "Anu", "Tara", "Dev"};
        for (int i = 0; i < n.length - 1; i++)      // selection sort
        {
            int p = i;
            for (int j = i + 1; j < n.length; j++)
                if (n[j].compareTo(n[p]) < 0)
                    p = j;
            String t = n[i];  n[i] = n[p];  n[p] = t;
        }
        for (String x : n)
            System.out.print(x + " ");
    }
}
```

```
Output
------------------------------
Anu Dev Rohan Tara
```



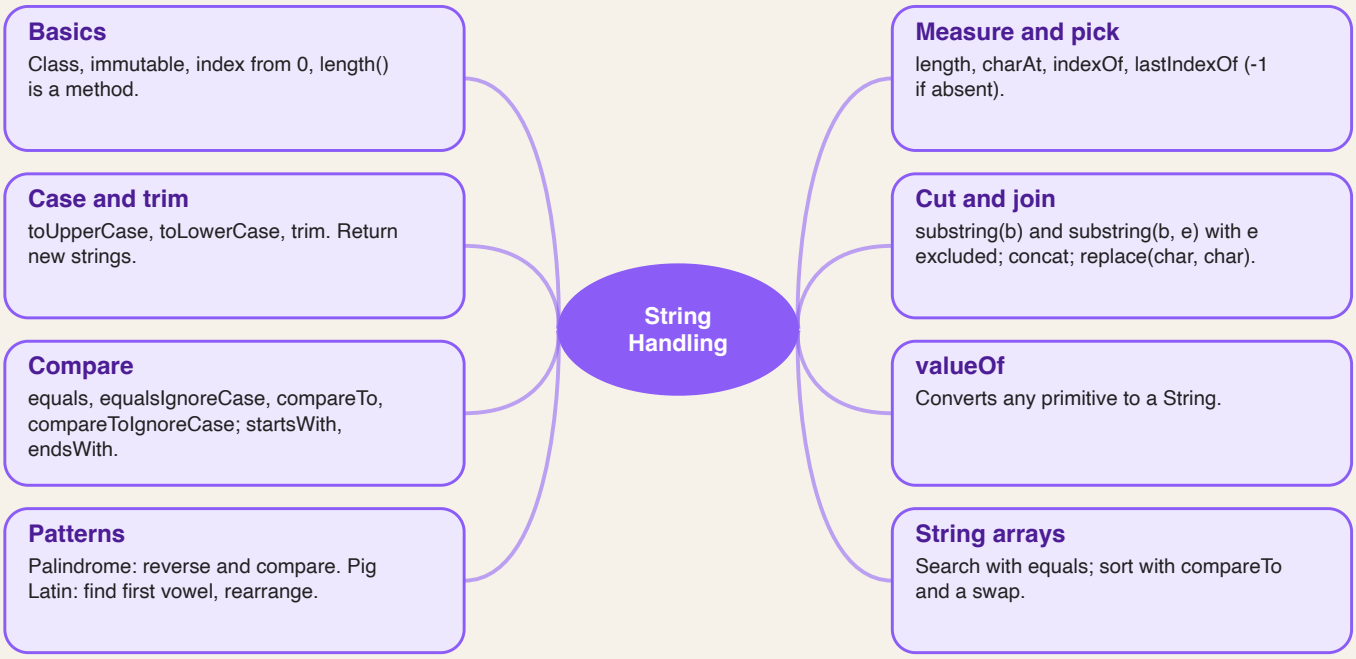
REAL LIFE

Phone contact lists, library catalogues and the Class 10 merit list all use exactly this idea: compare two pieces of text and swap them into order.

Common Mistakes

MISTAKE	FIX
`s.length` without brackets	String length is a method: `s.length()`. Array length has no brackets.
`s[0]` to get a character	Strings are not arrays. Use `s.charAt(0)`.
`if (s1 == s2)` for text	Use `s1.equals(s2)`.
`s.replace("a", 'b')`	The syllabus form takes two chars: `replace('a', 'b')`.
`s.charAt(s.length())`	Last index is `length() - 1`.
Forgetting to store the result	`s = s.trim();`

Mind Map Recap



HOT Questions

HOT QUESTION

What does `"BLUEJ".substring(1, 4).toLowerCase().indexOf('e')` return? (Answer: substring gives "LUE"; lower case "lue"; 'e' is at index 2.)

HOT QUESTION

Is `"abc".compareTo("abd")` equal to -1? Why? (Answer: yes, 'c' (99) minus 'd' (100) is -1.)

HOT QUESTION

A program prints `s.trim().length()` as 5 and `s.length()` as 9. How many space characters were removed? (Answer: 4, the leading and trailing ones.)

09

CHAPTER · COMPUTER APPLICATIONS

50 Must-Know Programs

Fifty programs in seven groups, each written from scratch and tested. Learn the pattern behind each group and new questions stop being new.



IN THIS CHAPTER

- Number programs
- Series
- Menus and conditions
- Patterns
- Arrays
- Strings
- Classes

These 50 programs are the ones the board keeps circling back to, grouped by topic. Every program here was compiled and run, and the output shown is the real output. **Do not memorise them.** Read the idea, then try writing the program yourself before looking at the code. Almost every program is a mix of the same few moves: extract digits, loop to a limit, count with a flag, swap, compare.



SIDEKICK TIP

Before any program, write down: **input** (what, which type), **process** (which loop or formula), **output** (what to print). That is the Variable Description Table the teacher wants in the Internal Assessment as well.

GROUP	PROGRAMS	SKILLS
A. Number programs	1 to 12	while loops, % and /, flags
B. Series	13 to 16	for loops, double, factorial
C. Conditionals and menus	17 to 22	if-else, switch, Math
D. Patterns	23 to 27	nested for loops
E. Arrays	28 to 37	1D, 2D, sorting
F. Strings	38 to 46	charAt, substring, compareTo
G. Classes and objects	47 to 50	constructors, methods, encapsulation

A. Number Programs



KEY POINT

Digit extraction pattern: `d = n % 10;` gives the last digit, `n = n / 10;` removes it. Wrap both in `while (n > 0)`. Keep a copy of the original number if you need it later.

Program 1: Armstrong number

A number is Armstrong if the sum of each digit raised to the power of the number of digits equals the number ($153 = 1^3 + 5^3 + 3^3$).

```
import java.util.*;
public class Armstrong
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), t = n, sum = 0;
        int d = String.valueOf(n).length();          // digit count
        while (t > 0)
        {
            int r = t % 10;
            sum += (int) Math.pow(r, d);
            t /= 10;
        }
        System.out.println(sum == n ? n + " is an Armstrong number"
                                    : n + " is not an Armstrong number");
    }
}
```

```
Input: 9474
-----------------------------
9474 is an Armstrong number
```

Idea: 9474 has 4 digits: $9^4 + 4^4 + 7^4 + 4^4 = 6561 + 256 + 2401 + 256 = 9474$.

Program 2: Perfect number

A number is perfect if the sum of its factors (excluding itself) equals the number. 6 = 1 + 2 + 3.

```
import java.util.*;
public class Perfect
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), s = 0;
        for (int i = 1; i < n; i++)
            if (n % i == 0)
                s += i;
        System.out.println("Sum of factors = " + s);
        System.out.println(s == n ? "Perfect" : "Not perfect");
    }
}
```

```
Input: 28
-----------------------------
Sum of factors = 28
Perfect
```

Idea: Test every i from 1 to n-1 as a possible factor (n % i == 0).

Program 3: Prime number (by counting factors)

A prime has exactly two factors: 1 and itself.

```
import java.util.*;
public class PrimeCheck
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), c = 0;
        for (int i = 1; i ≤ n; i++)
            if (n % i == 0)
                c++;
        if (c == 2)
            System.out.println(n + " is prime");
        else
            System.out.println(n + " is not prime (factors: " + c + ")");
    }
}
```

```
Input: 91
------------------------------
91 is not prime (factors: 4)
```

Idea: 91 = 7 × 13, so it has 4 factors (1, 7, 13, 91).

Program 4: Palindrome number

The number equals its reverse (12521, 4004).

```
import java.util.*;
public class PalNum
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), t = n, rev = 0;
        while (t > 0)
        {
            rev = rev * 10 + t % 10;
            t /= 10;
        }
        System.out.println("Reverse = " + rev);
        if (rev == n)
            System.out.println("Palindrome number");
        else
            System.out.println("Not a palindrome number");
    }
}
```

```
Input: 12521
------------------------------
Reverse = 12521
Palindrome number
```

Program 5: Digit count, sum, product and reverse

Show four facts about a number at once.

```
import java.util.*;
public class DigitFacts
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), t = n;
        int cnt = 0, sum = 0, prod = 1, rev = 0;
        while (t > 0)
        {
            int d = t % 10;
            cnt++;
            sum += d;
            prod *= d;
            rev = rev * 10 + d;
            t /= 10;
        }
        System.out.println("Digits=" + cnt + " Sum=" + sum);
        System.out.println("Product=" + prod + " Reverse=" + rev);
    }
}
```

```
Input: 4052
----------------------------
Digits=4 Sum=11
Product=0 Reverse=2504
```

Idea: Because 4052 contains a 0, the product is 0. Reversing 4050 would print 504, as a number cannot keep a leading zero.

Program 6: Factorial

n! = 1 × 2 × 3 x ... x n. Use `long` because factorials grow very fast (an int overflows after 12!).

```
import java.util.*;
public class Factorial
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long f = 1;
        for (int i = 2; i <= n; i++)
            f *= i;
        System.out.println(n + "! = " + f);
    }
}
```

```
Input: 15
----------------------------
15! = 1307674368000
```

Program 7: Fibonacci series

Each term is the sum of the previous two: 0, 1, 1, 2, 3, 5 ...

```
import java.util.*;
public class Fibonacci
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a = 0, b = 1;
        for (int i = 1; i ⩽ n; i++)
        {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
        System.out.println();
    }
}
```

```
Input: 10
-----------------------------
0 1 1 2 3 5 8 13 21 34
```

Idea: Slide the pair (a, b) forward: new a = old b, new b = old a + old b.

Program 8: GCD (HCF) and LCM

Euclid's method: replace the pair (a, b) by (b, a % b) until b is 0. Then LCM = a x b / GCD.

```
import java.util.*;
public class GcdLcm
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int x = sc.nextInt(), y = sc.nextInt();
        int a = x, b = y;
        while (b ≠ 0)
        {
            int t = a % b;
            a = b;
            b = t;
        }
        System.out.println("GCD = " + a);
        System.out.println("LCM = " + (x * y / a));
    }
}
```

```
Input: 36 48
-----------------------------
GCD = 12
LCM = 144
```

Program 9: Automorphic number

A number whose square ends with the number itself (76 → 5776).

```
import java.util.*;
public class Automorphic
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int sq = n * n;
        int p = (int) Math.pow(10, String.valueOf(n).length());
        System.out.println("Square = " + sq);
        if (sq % p == n)
            System.out.println("Automorphic");
        else
            System.out.println("Not automorphic");
    }
}
```

```
Input: 76
-----------------------------
Square = 5776
Automorphic
```

Idea: `sq % 10^digits` keeps only the last few digits of the square.

Program 10: Neon number

The sum of the digits of the **square** equals the number (9 → 81 → 8 + 1 = 9).

```
import java.util.*;
public class Neon
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int sq = n * n, s = 0;
        while (sq > 0)
        {
            s += sq % 10;
            sq /= 10;
        }
        System.out.println(s == n ? "Neon number" : "Not a neon number");
    }
}
```

```
Input: 9
-----------------------------
Neon number
```

Program 11: Harshad (Niven) number

A number divisible by the sum of its own digits (156: 1 + 5 + 6 = 12 and 156 / 12 = 13).

```
import java.util.*;
public class Harshad
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), t = n, s = 0;
        while (t != 0)
        {
            s += t % 10;
            t /= 10;
        }
        System.out.println("Digit sum = " + s);
        if (n % s == 0)
            System.out.println("Harshad number");
        else
            System.out.println("Not a Harshad number");
    }
}
```

```
Input: 156
-----------------------------
Digit sum = 12
Harshad number
```

Program 12: Spy number

The sum of the digits equals the product of the digits (1124: 1+1+2+4 = 8 and 1×1×2×4 = 8).

```
import java.util.*;
public class Spy
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), t = n, s = 0, p = 1;
        while (t > 0)
        {
            int d = t % 10;
            s += d;
            p *= d;
            t /= 10;
        }
        System.out.println("Sum=" + s + " Product=" + p);
        System.out.println(s == p ? "Spy number" : "Not a spy number");
    }
}
```

```
Input: 1124
-----------------------------
Sum=8 Product=8
Spy number
```

B. Series



SIDEKICK TIP

Series recipe. (1) Write the *k*th term as a formula. (2) Loop *k* from 1 to *n*. (3) Add the term to a `double` accumulator that starts at 0. Remember `1/k` is zero for k > 1 unless one operand is a double: write `1.0/k`.

Program 13: Sum of 1 + 1/2 + 1/3 + ... + 1/n

Harmonic series.

```
import java.util.*;
public class Harmonic
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        double s = 0;
        for (int k = 1; k ⩽ n; k++)
            s += 1.0 / k;
        System.out.println("Sum = " + s);
    }
}
```

```
Input: 5
-----------------------------
Sum = 2.283333333333333
```

Idea: Writing `1 / k` would give integer division. `1.0 / k` keeps the decimals.

Program 14: Sum of x/1! + x²/2! + x³/3! + ... to n terms

Keep a running power of x and a running factorial. Reuse the previous values instead of recomputing.

```
import java.util.*;
public class ExpSeries
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double x = sc.nextDouble();
        int n = sc.nextInt();
        double s = 0, pw = 1, f = 1;
        for (int k = 1; k <= n; k++)
        {
            pw *= x;            // x^k
            f *= k;             // k!
            s += pw / f;
        }
        System.out.println("Sum = " + s);
    }
}
```

```
Input: 2 4
----------------------------
Sum = 6.0
```

Program 15: Sum of 1! + 2! + 3! + ... + n!

Nested loops: the inner one builds each factorial.

```
import java.util.*;
public class FactSeries
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long total = 0;
        for (int i = 1; i <= n; i++)
        {
            long f = 1;
            for (int j = 1; j <= i; j++)
                f *= j;
            total += f;
        }
        System.out.println("Sum = " + total);
    }
}
```

```
Input: 5
----------------------------
Sum = 153
```

Idea: 1 + 2 + 6 + 24 + 120 = 153.

Program 16: Alternating series 1 - 2 + 3 - 4 + ... n terms

Flip a sign variable after every term.

```
import java.util.*;
public class Alternate
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), s = 0, sign = 1;
        for (int k = 1; k ⩽ n; k++)
        {
            s += sign * k;
            sign = -sign;
        }
        System.out.println("Sum = " + s);
    }
}
```

```
Input: 9
-----------------------------
Sum = 5
```

C. Conditionals and Menu-Driven Programs

Program 17: Menu driven: area of a shape

Show a menu, read the choice and use `switch` with a `default` case.

```
import java.util.*;
public class ShapeMenu
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Circle  2. Rectangle  3. Triangle");
        System.out.print("Choice: ");
        int ch = sc.nextInt();
        switch (ch)
        {
            case 1:
                double r = sc.nextDouble();
                System.out.println("Area = " + 3.14159 * r * r);
                break;
            case 2:
                double l = sc.nextDouble(), b = sc.nextDouble();
                System.out.println("Area = " + l * b);
                break;
            case 3:
                double bs = sc.nextDouble(), h = sc.nextDouble();
                System.out.println("Area = " + 0.5 * bs * h);
                break;
            default:
                System.out.println("Wrong choice");
        }
    }
}
```

```
Input: 2 | 7.5 4
-----------------------------
1. Circle  2. Rectangle  3. Triangle
Choice: Area = 30.0
```

Program 18: Roots of a quadratic equation

For $ax^2 + bx + c = 0$ the discriminant $D = b^2 - 4ac$ decides the nature of the roots.

```
import java.util.*;
public class Quadratic
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double a = sc.nextDouble(), b = sc.nextDouble(), c = sc.nextDouble();
        double d = b * b - 4 * a * c;
        if (d > 0)
        {
            double r1 = (-b + Math.sqrt(d)) / (2 * a);
            double r2 = (-b - Math.sqrt(d)) / (2 * a);
            System.out.println("Real and distinct: " + r1 + " , " + r2);
        }
        else if (d == 0)
            System.out.println("Equal roots: " + (-b / (2 * a)));
        else
            System.out.println("No real roots (D = " + d + ")");
    }
}
```

```
Input: 1 -5 6
-----------------------------
Real and distinct: 3.0 , 2.0
```

Program 19: Leap year

A year is a leap year if divisible by 400, or divisible by 4 but not by 100.

```
import java.util.*;
public class LeapYear
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int y = sc.nextInt();
        boolean leap = (y % 400 == 0) || (y % 4 == 0 && y % 100 != 0);
        System.out.println(y + (leap ? " is a leap year"
                                     : " is not a leap year"));
    }
}
```

```
Input: 2100
-----------------------------
2100 is not a leap year
```

Idea: 2100 is divisible by 100 but not by 400, so it is **not** a leap year.

Program 20: Electricity bill with slabs

First 100 units at Rs 2.50, the next 100 at Rs 4.00, the rest at Rs 6.00.

```
import java.util.*;
public class Bill
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int u = sc.nextInt();
        double amt;
        if (u ⩽ 100)
            amt = u * 2.5;
        else if (u ⩽ 200)
            amt = 100 * 2.5 + (u - 100) * 4.0;
        else
            amt = 100 * 2.5 + 100 * 4.0 + (u - 200) * 6.0;
        System.out.println("Units = " + u + ", Bill = Rs " + amt);
    }
}
```

```
Input: 260
------------------------------
Units = 260, Bill = Rs 1010.0
```

Idea: 250 + 400 + 60 units x 6 = 1010.

Program 21: Type of triangle

First check validity (the sum of any two sides must exceed the third), then classify.

```
import java.util.*;
public class TriType
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt(), b = sc.nextInt(), c = sc.nextInt();
        if (a + b ⩽ c || b + c ⩽ a || a + c ⩽ b)
            System.out.println("Not a triangle");
        else if (a == b && b == c)
            System.out.println("Equilateral");
        else if (a == b || b == c || a == c)
            System.out.println("Isosceles");
        else
            System.out.println("Scalene");
    }
}
```

```
Input: 5 5 8
------------------------------
Isosceles
```

Program 22: Calculator with a character operator

Read two numbers and an operator and use `switch` on the `char`.

```
import java.util.*;
public class Calc2
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        double a = sc.nextDouble(), b = sc.nextDouble();
        char op = sc.next().charAt(0);
        switch (op)
        {
            case '+': System.out.println(a + b); break;
            case '-': System.out.println(a - b); break;
            case '*': System.out.println(a * b); break;
            case '/':
                if (b == 0) System.out.println("Cannot divide by zero");
                else        System.out.println(a / b);
                break;
            default: System.out.println("Invalid operator");
        }
    }
}
```

```
Input: 15 4 | /
------------------------------
3.75
```

D. Patterns



KEY POINT

Row number *i* usually decides: how many spaces (`n − i`), how many symbols (`i` or `2i − 1`) and which value is printed. Always finish a row with `System.out.println();`.

Program 23: Number triangle

Row *i* prints the numbers from 1 to *i*.

```
public class NumTri
{
    public static void main(String[] args)
    {
        int n = 5;
        for (int i = 1; i ⩽ n; i++)
        {
            for (int j = 1; j ⩽ i; j++)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

```
Output
-----------------------------
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Program 24: Star pyramid

Row *i* has `n − i` spaces then `2i − 1` stars.

```
public class Pyramid
{
    public static void main(String[] args)
    {
        int n = 5;
        for (int i = 1; i ≤ n; i++)
        {
            for (int s = 1; s ≤ n - i; s++)
                System.out.print(" ");
            for (int k = 1; k ≤ 2 * i - 1; k++)
                System.out.print("*");
            System.out.println();
        }
    }
}
```

```
Output
-----------------------------
    *
   ***
  *****
 *******
*********
```

Program 25: Inverted countdown triangle

Row *i* prints from 5 down to `i`.

```
public class Countdown
{
    public static void main(String[] args)
    {
        int n = 5;
        for (int i = 1; i ≤ n; i++)
        {
            for (int j = n; j ≥ i; j--)
                System.out.print(j + " ");
            System.out.println();
        }
    }
}
```

```
Output
-----------------------------
5 4 3 2 1
5 4 3 2
5 4 3
5 4
5
```

Program 26: Diamond of stars

An upper pyramid followed by an inverted one without repeating the widest row.

```
public class Diamond
{
    public static void main(String[] args)
    {
        int n = 4;
        for (int i = 1; i ⩽ n; i++)                // upper half
        {
            for (int s = 1; s ⩽ n - i; s++) System.out.print(" ");
            for (int k = 1; k ⩽ 2 * i - 1; k++) System.out.print("*");
            System.out.println();
        }
        for (int i = n - 1; i ⩾ 1; i--)            // lower half
        {
            for (int s = 1; s ⩽ n - i; s++) System.out.print(" ");
            for (int k = 1; k ⩽ 2 * i - 1; k++) System.out.print("*");
            System.out.println();
        }
    }
}
```

```
Output
-----------------------------
   *
  ***
 *****
*******
 *****
  ***
   *
```

Program 27: Floyd's triangle

Consecutive numbers laid out in rows of growing length, using one counter that never resets.

```
public class Floyd
{
    public static void main(String[] args)
    {
        int n = 5, c = 1;
        for (int i = 1; i ⩽ n; i++)
        {
            for (int j = 1; j ⩽ i; j++)
            {
                System.out.print(c + "\t");
                c++;
            }
            System.out.println();
        }
    }
}
```

```
Output
------------------------------
1
2       3
4       5       6
7       8       9       10
11      12      13      14      15
```

E. Arrays



KEY POINT

Array problems come in three shapes: **scan** (one loop, keep a running answer), **compare pairs** (nested loop) and **rearrange** (swap with a temporary variable).

Program 28: Largest and second largest

Keep two variables. When a bigger value arrives, the old largest becomes the second largest.

```
import java.util.*;
public class SecondMax
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] a = new int[6];
        for (int i = 0; i < 6; i++)
            a[i] = sc.nextInt();
        int max = a[0], second = Integer.MIN_VALUE;
        for (int i = 1; i < a.length; i++)
        {
            if (a[i] > max)
            {
                second = max;
                max = a[i];
            }
            else if (a[i] > second && a[i] != max)
                second = a[i];
        }
        System.out.println("Largest = " + max + ", second = " + second);
    }
}
```

```
Input: 14 52 9 52 31 40
------------------------------
Largest = 52, second = 40
```

Program 29: Reverse an array in place

Swap the first with the last, the second with the second last, and stop at the middle.

```
public class RevArray
{
    public static void main(String[] args)
    {
        int[] a = {10, 20, 30, 40, 50, 60, 70};
        for (int i = 0, j = a.length - 1; i < j; i++, j--)
        {
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
        for (int x : a)
            System.out.print(x + " ");
        System.out.println();
    }
}
```

```
Output
------------------------------
70 60 50 40 30 20 10
```

Program 30: Frequency of each element

For every element, count how many times it occurs. Mark counted ones in a boolean array so they are not reported twice.

```
public class Frequency
{
    public static void main(String[] args)
    {
        int[] a = {4, 7, 4, 2, 7, 4, 9};
        boolean[] done = new boolean[a.length];
        for (int i = 0; i < a.length; i++)
        {
            if (done[i]) continue;
            int c = 1;
            for (int j = i + 1; j < a.length; j++)
            {
                if (a[j] == a[i])
                {
                    c++;
                    done[j] = true;
                }
            }
            System.out.println(a[i] + " occurs " + c + " time(s)");
        }
    }
}
```

```
Output
----------------------------
4 occurs 3 time(s)
7 occurs 2 time(s)
2 occurs 1 time(s)
9 occurs 1 time(s)
```

Idea: A parallel `boolean` array remembers which positions were already counted.

Program 31: Merge two sorted arrays

Walk through both arrays with two pointers, always copying the smaller current element.

```
public class MergeSorted
{
    public static void main(String[] args)
    {
        int[] a = {2, 9, 15, 20};
        int[] b = {1, 9, 12, 30, 40};
        int[] m = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length)
            m[k++] = (a[i] ⩽ b[j]) ? a[i++] : b[j++];
        while (i < a.length) m[k++] = a[i++];
        while (j < b.length) m[k++] = b[j++];
        for (int x : m)
            System.out.print(x + " ");
        System.out.println();
    }
}
```

```
Output
------------------------------
1 2 9 9 12 15 20 30 40
```

Program 32: Separate even and odd elements

Copy the evens to the front of a new array and the odds after them, keeping the original order.

```
public class EvenOdd
{
    public static void main(String[] args)
    {
        int[] a = {5, 8, 12, 7, 3, 10, 6};
        int[] r = new int[a.length];
        int k = 0;
        for (int i = 0; i < a.length; i++)       // evens first
            if (a[i] % 2 == 0)
                r[k++] = a[i];
        int evens = k;
        for (int i = 0; i < a.length; i++)       // then odds
            if (a[i] % 2 ≠ 0)
                r[k++] = a[i];
        for (int x : r)
            System.out.print(x + " ");
        System.out.println("(evens: " + evens + ")");
    }
}
```

```
Output
------------------------------
8 12 10 6 5 7 3 (evens: 4)
```

Program 33: Insert a value at a given position

Shift everything from that position one place right (start from the end), then drop the new value in.

```
public class InsertAt
{
    public static void main(String[] args)
    {
        int[] a = new int[7];
        int n = 5;
        int[] init = {11, 22, 33, 44, 55};
        for (int i = 0; i < n; i++)
            a[i] = init[i];
        int pos = 2, val = 99;
        for (int i = n; i > pos; i--)       // shift right
            a[i] = a[i - 1];
        a[pos] = val;
        n++;
        for (int i = 0; i < n; i++)
            System.out.print(a[i] + " ");
        System.out.println();
    }
}
```

```
Output
------------------------------
11 22 99 33 44 55
```

Program 34: Add two matrices

Matrices of the same order are added cell by cell.

```
public class MatAdd
{
    public static void main(String[] args)
    {
        int[][] p = { {1, 2, 3}, {4, 5, 6} };
        int[][] q = { {9, 8, 7}, {6, 5, 4} };
        int[][] s = new int[2][3];
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 3; j++)
            {
                s[i][j] = p[i][j] + q[i][j];
                System.out.print(s[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

```
Output
------------------------------
10 10 10
10 10 10
```

Program 35: Transpose of a matrix

Rows become columns: `t[j][i] = m[i][j]`.

```
public class Transpose
{
    public static void main(String[] args)
    {
        int[][] m = { {1, 2, 3}, {4, 5, 6} };
        int[][] t = new int[3][2];
        for (int i = 0; i < 2; i++)
            for (int j = 0; j < 3; j++)
                t[j][i] = m[i][j];
        for (int i = 0; i < 3; i++)
        {
            for (int j = 0; j < 2; j++)
                System.out.print(t[i][j] + " ");
            System.out.println();
        }
    }
}
```

```
Output
----------------------------
1 4
2 5
3 6
```

Idea: The new matrix has the dimensions swapped: a 2 × 3 matrix becomes 3 × 2.

Program 36: Check for a symmetric matrix

A square matrix is symmetric if `m[i][j] == m[j][i]` for every cell.

```
public class Symmetric
{
    public static void main(String[] args)
    {
        int[][] m = { {1, 7, 3}, {7, 4, 5}, {3, 5, 9} };
        boolean sym = true;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                if (m[i][j] ≠ m[j][i])
                    sym = false;
        System.out.println(sym ? "Symmetric matrix" : "Not symmetric");
    }
}
```

```
Output
----------------------------
Symmetric matrix
```

Program 37: Sum of boundary elements

A cell is on the boundary if it lies in the first or last row, or the first or last column.

```
public class Boundary
{
    public static void main(String[] args)
    {
        int[][] m = { {1, 2, 3, 4},
                      {5, 6, 7, 8},
                      {9, 1, 2, 3} };
        int s = 0, r = m.length, c = m[0].length;
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++)
                if (i == 0 || i == r - 1 || j == 0 || j == c - 1)
                    s += m[i][j];
        System.out.println("Boundary sum = " + s);
    }
}
```

```
Output
------------------------------
Boundary sum = 38
```

Idea: Here only 6 and 7 are interior. Total of all = 51, minus 13 = 38.

F. Strings

Program 38: Reverse every word of a sentence

Cut each word with `indexOf(' ')`, reverse it, and rebuild the sentence.

```
import java.util.*;
public class RevWords
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine() + " ", out = "";
        while (s.length() > 0)
        {
            int sp = s.indexOf(' ');
            String w = s.substring(0, sp), r = "";
            for (int i = w.length() - 1; i >= 0; i--)
                r += w.charAt(i);
            out += r + " ";
            s = s.substring(sp + 1);
        }
        System.out.println(out.trim());
    }
}
```

```
Input: learn java daily
------------------------------
nrael avaj yliad
```

Program 39: Anagram check

Two words are anagrams if they have the same letters in a different order. Sort the letters of each and compare.

```
import java.util.*;
public class Anagram
{
    static String sortLetters(String w)
    {
        char[] c = w.toLowerCase().toCharArray();
        for (int i = 0; i < c.length - 1; i++)
            for (int j = 0; j < c.length - 1 - i; j++)
                if (c[j] > c[j + 1])
                {
                    char t = c[j];
                    c[j] = c[j + 1];
                    c[j + 1] = t;
                }
        return new String(c);
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String a = sc.next(), b = sc.next();
        if (sortLetters(a).equals(sortLetters(b)))
            System.out.println("Anagrams");
        else
            System.out.println("Not anagrams");
    }
}
```

```
Input: Listen | Silent
------------------------------
Anagrams
```

Idea: `toCharArray()` turns a String into a char array so that it can be sorted. It is a handy method beyond the syllabus list.

Program 40: Capitalise the first letter of each word

After a space, the next letter must be a capital; all other letters become small.

```
import java.util.*;
public class TitleCase
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine(), out = "";
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (i == 0 || s.charAt(i - 1) == ' ')
                out += Character.toUpperCase(c);
            else
                out += Character.toLowerCase(c);
        }
        System.out.println(out);
    }
}
```

```
Input: wELcome tO thE ICSE exam
------------------------------
Welcome To The Icse Exam
```

Program 41: Frequency of each letter

Use an int array of size 26: letter c goes to index `c - 'a'`.

```
import java.util.*;
public class LetterFreq
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().toLowerCase();
        int[] f = new int[26];
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (c >= 'a' && c <= 'z')
                f[c - 'a']++;
        }
        for (int i = 0; i < 26; i++)
            if (f[i] > 0)
                System.out.print((char) ('a' + i) + "=" + f[i] + " ");
        System.out.println();
    }
}
```

```
Input: Banana Bread
------------------------------
a=4 b=2 d=1 e=1 n=2 r=1
```

Program 42: Caesar cipher (shift by 3)

Move each letter 3 places ahead in the alphabet and wrap around from Z to A.

```
import java.util.*;
public class Caesar
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine(), out = "";
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (Character.isUpperCase(c))
                out += (char) ('A' + (c - 'A' + 3) % 26);
            else if (Character.isLowerCase(c))
                out += (char) ('a' + (c - 'a' + 3) % 26);
            else
                out += c;
        }
        System.out.println(out);
    }
}
```

```
Input: Xyz Abc, Java!
-----------------------------
Abc Def, Mdyd!
```

Idea: `%` 26 wraps X, Y, Z back to A, B, C.

Program 43: Abbreviate a name

Ayush Kumar Tripathi becomes A. K. Tripathi: initials for every word except the last.

```
import java.util.*;
public class Abbrev
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String name = sc.nextLine().trim();
        int last = name.lastIndexOf(' ');
        String out = "";
        for (int i = 0; i < last; i++)
        {
            if (i == 0 || name.charAt(i - 1) == ' ')
                out += name.charAt(i) + ". ";
        }
        out += name.substring(last + 1);
        System.out.println(out);
    }
}
```

```
Input: Ananya Rao Mehta
-----------------------------
A. R. Mehta
```

Program 44: Is the word in ascending alphabetical order?

Every letter must be not smaller than the one before it (ALMOST passes, ALMOND fails).

```
import java.util.*;
public class AscWord
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String w = sc.next().toUpperCase();
        boolean asc = true;
        for (int i = 1; i < w.length(); i++)
            if (w.charAt(i) < w.charAt(i - 1))
                asc = false;
        System.out.println(w + (asc ? " is in ascending order"
                                    : " is not in ascending order"));
    }
}
```

```
Input: almost
-----------------------------
ALMOST is in ascending order
```

Program 45: Remove duplicate letters

Keep a letter only if it has not already been kept (checked with `indexOf` on the result).

```
import java.util.*;
public class Dedup
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.next(), out = "";
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (out.indexOf(c) == -1)
                out += c;
        }
        System.out.println(out);
    }
}
```

```
Input: programming
-----------------------------
progamin
```

Program 46: Sum of digits hidden in text

Pick the digit characters out of a mixed string and add their values.

```
import java.util.*;
public class HiddenSum
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int sum = 0, count = 0;
        for (int i = 0; i < s.length(); i++)
        {
            char c = s.charAt(i);
            if (Character.isDigit(c))
            {
                sum += c - '0';
                count++;
            }
        }
        System.out.println(count + " digits, sum = " + sum);
    }
}
```

```
Input: Room 4B, floor 27, code 9Z
-----------------------------
4 digits, sum = 22
```

G. Classes and Objects

ICSE class-based questions give a class name, the data members and the member functions, and ask you to complete them. Use this order: **variables, constructor, methods, main**.

Program 47: Employee class: constructor and salary

Class with private data, a parameterised constructor, a method that computes gross pay and a method that displays it.

```
public class Employee
{
    private String name;
    private double basic, gross;

    Employee(String n, double b)
    {
        name = n;
        basic = b;
    }
    void compute()
    {
        double hra = 0.20 * basic, da = 0.50 * basic;
        gross = basic + hra + da;
    }
    void display()
    {
        System.out.println(name + " earns " + gross);
    }
    public static void main(String[] args)
    {
        Employee e = new Employee("Kavya", 20000);
        e.compute();
        e.display();
    }
}
```

```
Output
------------------------------
Kavya earns 34000.0
```

Program 48: Overloaded methods: area and volume

Same name, different parameter lists.

```
public class Shapes
{
    double calc(double s)                 // area of a square
    {
        return s * s;
    }
    double calc(double l, double b)       // area of a rectangle
    {
        return l * b;
    }
    double calc(double l, double b, double h)   // volume of a cuboid
    {
        return l * b * h;
    }
    public static void main(String[] args)
    {
        Shapes x = new Shapes();
        System.out.println(x.calc(6));
        System.out.println(x.calc(6, 4));
        System.out.println(x.calc(6, 4, 2.5));
    }
}
```

```
Output
------------------------------
36.0
24.0
60.0
```

Program 49: Number class with a helper method

A class that stores a number and tells whether it is prime and what its digit sum is. A method can call another method of the same class.

```
import java.util.*;
public class NumInfo
{
    int n;

    NumInfo(int n)
    {
        this.n = n;
    }
    int digitSum()
    {
        int t = n, s = 0;
        while (t > 0)
        {
            s += t % 10;
            t /= 10;
        }
        return s;
    }
    boolean isPrime()
    {
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) return false;
        return true;
    }
    void show()
    {
        System.out.println(n + ": digit sum " + digitSum()
                           + ", prime = " + isPrime());
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        new NumInfo(sc.nextInt()).show();
    }
}
```

```
Input: 2027
----------------------------
2027: digit sum 11, prime = true
```

Program 50: Library book: static counter and object array

Each book counts itself in a static variable; an array of objects is searched by title.

```
public class Library
{
    static int total = 0;          // shared by every book
    String title;
    int copies;

    Library(String t, int c)
    {
        title = t;
        copies = c;
        total++;
    }
    public static void main(String[] args)
    {
        Library[] shelf = {
            new Library("Wings of Fire", 3),
            new Library("Ikigai", 1),
            new Library("Malgudi Days", 4)
        };
        String find = "Ikigai";
        for (int i = 0; i < shelf.length; i++)
            if (shelf[i].title.equalsIgnoreCase(find))
                System.out.println(find + ": " + shelf[i].copies + " copy");
        System.out.println("Books created: " + Library.total);
    }
}
```

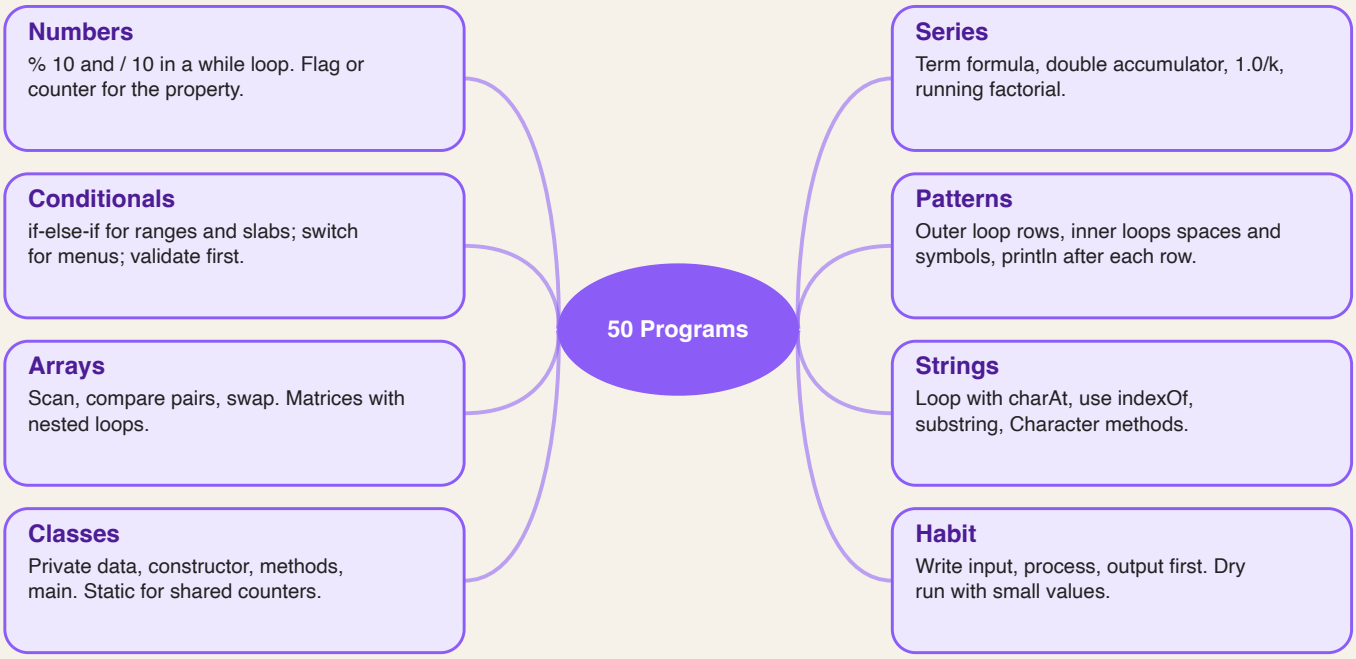
```
Output
-----------------------------
Ikigai: 1 copy
Books created: 3
```



FUN FACT

You have now seen the main ideas behind almost every ICSE Java program: digit loops, flag variables, counting, swapping, two-pointer scans, nested-loop tables and character arithmetic. New questions are remixes of these.

Mind Map Recap



10

CHAPTER · COMPUTER APPLICATIONS

Practice Paper: Exam-Style Questions with Answers

Fifty minutes of focused practice. Four question types in the board's style, all newly written, with fully worked answers.



IN THIS CHAPTER

- Predict the output
- Find the error
- Write the program
- Multiple choice
- Answer key

This chapter is a mock paper in four sections. Every question is original. Cover the answers, attempt on paper, then compare. The full answers start after Section D. Work in the board's style: show every step of a dry run, write the variable list before a program, and never leave a question blank.



SIDEKICK TIP

Time plan for a 2-hour paper (80 marks in Section B): Section A (compulsory short answers, 40 marks) 50 minutes, then 4 of 6 programs in Section B, about 15 minutes each, leaving 10 minutes for review. Check semicolons, braces and the case of every keyword before you move on.

Section A: Predict the Output (Dry Run on Paper)

Question 1

```
int s = 0;
for (int i = 1; i ⩽ 10; i++)
{
    if (i % 3 == 0) continue;
    if (i > 8) break;
    s += i;
}
System.out.println(s);
```

Question 2

```
String s = "Computer Science";
System.out.println(s.indexOf('e') + " " + s.lastIndexOf('e'));
System.out.println(s.substring(4, 8).toUpperCase());
System.out.println(s.charAt(s.length() - 1) + s.charAt(0));
System.out.println(s.replace('e', 'E').endsWith("cE"));
```

Question 3

```
int[] a = {5, 3, 8, 1};
for (int i = 0; i < a.length - 1; i++)
    a[i] = a[i] + a[i + 1];
for (int x : a)
    System.out.print(x + " ");
```

Question 4

```
int[][] m = new int[3][3];
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        m[i][j] = i * 3 + j;
int d = 0;
for (int i = 0; i < 3; i++)
    d += m[i][2 - i];
System.out.println(m[2][1] + " " + d);
```

Question 5

```
int x = 3;
switch (x)
{
    case 2: System.out.print("two ");
    case 3: System.out.print("three ");
    case 4: System.out.print("four "); break;
    case 5: System.out.print("five ");
    default: System.out.print("other");
}
```

Question 6

```
String a = "Java", b = "java";
System.out.println(a.compareTo(b));
System.out.println(a.compareToIgnoreCase(b));
System.out.println(a == b);
System.out.println(a.equalsIgnoreCase(b));
System.out.println(1 + 2 + a + 1 + 2);
```

Question 7

```
for (int i = 1; i ⩽ 4; i++)
{
    for (int j = 4; j ⩾ i; j--)
        System.out.print(j);
    System.out.println();
}
```

Question 8

```
char c = 'k';
int n = Integer.parseInt("12") + Integer.parseInt("8");
System.out.println(Character.toUpperCase(c));
System.out.println((char) (c + 2));
System.out.println(c + 2);
System.out.println(n);
System.out.println("12" + "8");
System.out.println(Character.isLetterOrDigit('_'));
```

Question 9

```
int a = 10, b = 4;
a += b++ * 2;
b -= --a % 5;
System.out.println(a + " " + b);
```

Section B: Find the Error

Question 1

```
int x = 5.5;
```

Question 2

```
int sign(int a)
{
    if (a > 0)
        return 1;
}
```

Question 3

```
public static void main(String[] args)
{
    int r = square(4);
}
int square(int n) { return n * n; }
```

Question 4

```
String s = "hello";
if (s.length > 3)
    System.out.println("long");
```

Question 5

```
class Pen
{
    void Pen() { colour = "blue"; }
}
...
Pen p = new Pen();
```

Question 6

```
char c = "A";
```

Question 7

```
int[] a = new int[3];
a[3] = 10;
```

Question 8

```
int total;
System.out.println(total + 5);
```

Section C: Write the Program

1. Accept a sentence. Display the number of words that begin with a capital letter, and print the longest word. Assume words are separated by a single space.

2. A taxi company charges Rs 50 for the first 2 km and Rs 12 per km after that, plus a waiting charge of Rs 1.50 per minute. Write a class **Taxi** with data members, a parameterised constructor, a method to compute the fare and a method to display it.

3. Accept 8 integers into an array. Using bubble sort, arrange them in descending order and display the array. Also display the second smallest value.

4. Write a program to fill a 4 × 4 matrix with the numbers 1 to 16 row by row, display it, and check whether the sum of the left diagonal equals the sum of the right diagonal.

5. Write a menu driven program: option 1 reads a number and prints whether it is a palindrome; option 2 reads a word and prints the number of vowels it has; any other option prints a message.

Section D: Multiple Choice

1. Which of these is a valid identifier?
(a) 2total (b) total_2 (c) class (d) my var

2. The default value of an uninitialised boolean instance variable is
(a) true (b) false (c) null (d) 0

3. What is the value of `10 / 4 * 2.0`?
(a) 5.0 (b) 4.0 (c) 5 (d) 4

4. Which method returns the smallest whole number not less than its argument?
(a) Math.floor (b) Math.round (c) Math.ceil (d) Math.rint

5. A constructor
(a) has a return type of void (b) has the same name as the class (c) must be static (d) is called by the programmer using the dot

6. Which is NOT a feature of a constructor?
(a) No return type (b) Can be overloaded (c) Is invoked automatically (d) Can return an int

7. Converting `Integer x = 7;` uses
(a) Unboxing (b) Autoboxing (c) Casting (d) Parsing

8. `Character.isLetterOrDigit('$')` returns
(a) true (b) false (c) 0 (d) error

9. Which access specifier gives the widest visibility?
(a) private (b) protected (c) public (d) default

10. A variable declared with `static` inside a class is a
(a) local variable (b) argument variable (c) class variable (d) instance variable

11. The statement `int[] a = new int[5];` creates an array whose last valid index is
(a) 5 (b) 4 (c) 6 (d) 0

12. Binary search requires the array to be
(a) of even length (b) sorted (c) of strings only (d) two dimensional

13. For a 3 × 3 matrix, the cells on the right diagonal satisfy
(a) i == j (b) i + j == 2 (c) i + j == 3 (d) i - j == 1

14. `"Programming".indexOf('m')` returns
(a) 6 (b) 7 (c) 5 (d) -1

15. `"computer".substring(2, 5)` returns
(a) mpu (b) mput (c) omp (d) mpute

16. Which statement about String is TRUE?
(a) It is a primitive type (b) It is mutable (c) It is a class and immutable (d) It is an array of int

17. `"Hello".compareTo("Help")` is
(a) negative (b) zero (c) positive (d) an error

18. Overloaded methods must differ in
(a) return type only (b) parameter list (c) method name (d) access specifier only

19. A pure method
(a) changes the object's state (b) does not change the object's state (c) always returns void (d) must be static

20. `Integer.parseInt("4a")` throws
(a) ArithmeticException (b) NumberFormatException (c) ArrayIndexOutOfBoundsException (d) No exception

Answers: Section A

Q1. i = 1, 2, 4, 5, 7, 8 are added (27). 3, 6 and 9 are skipped by continue; at i = 10 the break fires.

```
27
```

Q2. 'e' is first at index 6 and last at 15. substring(4, 8) is "uter". 'e' (101) + 'C' (67) = 168 as ints. After replace the ending is "cE", so true.

```
6 15
UTER
168
true
```

Q3. Each element takes the sum with its right neighbour using the already updated array: 5+3 = 8, 3+8 = 11, 8+1 = 9, last stays 1.

```
8 11 9 1
```

Q4. The matrix holds 0..8 row by row. m[2][1] = 7. Right diagonal: 2 + 4 + 6 = 12.

```
7 12
```

Q5. Execution enters at case 3 and, with no break, falls into case 4 where the break stops it.

```
three four
```

Q6. 'J' (74) - 'j' (106) = -32. Ignoring case gives 0. == compares references (false). 1+2 = 3 first, then the string joins: 3Java12.

```
-32
0
false
true
3Java12
```

Q7. Row i prints the digits from 4 down to i.

```
4321
432
43
4
```

Q8. 'k' is 107. Upper case K. (char)(107 + 2) is 'm'. Without the cast 109 prints. 12 + 8 = 20 but the two strings are joined to 128. The underscore is neither letter nor digit.

```
K
m
109
20
128
false
```

Q9. b++ * 2 = 4 * 2 = 8 and b becomes 5; a = 18. --a makes a 17, 17 % 5 = 2, so b = 5 - 2 = 3.

```
17 3
```

Answers: Section B

NO.	ERROR AND CORRECTION
1	Incompatible types: *possible lossy conversion from double to int*. Fix: `double x = 5.5;` or `int x = (int) 5.5;`
2	*Missing return statement*. The method returns nothing when a <= 0. Add `return 0;` (or an else branch).
3	Non-static method called from static `main` without an object. Make `square` static or call new `ClassName().square(4)`.
4	`length` is a method for strings. Write `s.length()`.
5	`void Pen()` is an ordinary method, not a constructor, so it never runs at `new Pen()`. Remove `void`. (The variable `colour` is also undeclared.)
6	Incompatible types: String cannot be converted to char. Use single quotes: `char c = 'A';`
7	Compiles, but at run time throws **ArrayIndexOutOfBoundsException**. Valid indexes are 0, 1 and 2.
8	*Variable total might not have been initialized*. A local variable must be given a value before use.

Answers: Section C (Model Programs)

Solution 1

```
import java.util.*;
public class W1
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine() + " ";
        int cap = 0;
        String longest = "";
        while (s.length() > 0)
        {
            int sp = s.indexOf(' ');
            String w = s.substring(0, sp);
            if (Character.isUpperCase(w.charAt(0)))
                cap++;
            if (w.length() > longest.length())
                longest = w;
            s = s.substring(sp + 1);
        }
        System.out.println("Capital words: " + cap);
        System.out.println("Longest word: " + longest);
    }
}
```

```
Input: Rohit and Meera visited Jaipur yesterday
------------------------------
Capital words: 3
Longest word: yesterday
```

Solution 2

```
public class Taxi
{
    private double km, wait, fare;

    Taxi(double km, double wait)
    {
        this.km = km;
        this.wait = wait;
    }
    void compute()
    {
        fare = 50;
        if (km > 2)
            fare += (km - 2) * 12;
        fare += wait * 1.5;
    }
    void display()
    {
        System.out.println("Distance: " + km + " km, waiting: " + wait + " min");
        System.out.println("Fare: Rs " + fare);
    }
    public static void main(String[] args)
    {
        Taxi t = new Taxi(9.5, 10);
        t.compute();
        t.display();
    }
}
```

```
Output
----------------------------
Distance: 9.5 km, waiting: 10.0 min
Fare: Rs 155.0
```

Solution 3

```
import java.util.*;
public class W3
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int[] a = new int[8];
        for (int i = 0; i < 8; i++)
            a[i] = sc.nextInt();
        for (int i = 0; i < 7; i++)
            for (int j = 0; j < 7 - i; j++)
                if (a[j] < a[j + 1])             // descending
                {
                    int t = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = t;
                }
        for (int x : a)
            System.out.print(x + " ");
        System.out.println();
        System.out.println("Second smallest: " + a[6]);
    }
}
```

```
Input: 23 7 41 16 9 35 2 28
-----------------------------
41 35 28 23 16 9 7 2
Second smallest: 7
```

Solution 4

```
public class W4
{
    public static void main(String[] args)
    {
        int[][] m = new int[4][4];
        int k = 1, left = 0, right = 0;
        for (int i = 0; i < 4; i++)
        {
            for (int j = 0; j < 4; j++)
            {
                m[i][j] = k++;
                System.out.print(m[i][j] + "\t");
            }
            System.out.println();
        }
        for (int i = 0; i < 4; i++)
        {
            left += m[i][i];
            right += m[i][3 - i];
        }
        System.out.println("Left = " + left + ", Right = " + right);
        System.out.println(left == right ? "Equal" : "Not equal");
    }
}
```

```
Output
------------------------------
1       2       3       4
5       6       7       8
9       10      11      12
13      14      15      16
Left = 34, Right = 34
Equal
```

Solution 5

```
import java.util.*;
public class W5
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("1. Palindrome number  2. Vowels in a word");
        int ch = sc.nextInt();
        switch (ch)
        {
            case 1:
                int n = sc.nextInt(), t = n, r = 0;
                while (t > 0)
                {
                    r = r * 10 + t % 10;
                    t /= 10;
                }
                System.out.println(r == n ? "Palindrome" : "Not palindrome");
                break;
            case 2:
                String w = sc.next().toLowerCase();
                int v = 0;
                for (int i = 0; i < w.length(); i++)
                    if ("aeiou".indexOf(w.charAt(i)) != -1)
                        v++;
                System.out.println("Vowels = " + v);
                break;
            default:
                System.out.println("Invalid option");
        }
    }
}
```

```
Input: 2 | education
----------------------------
1. Palindrome number  2. Vowels in a word
Vowels = 5
```

Answers: Section D

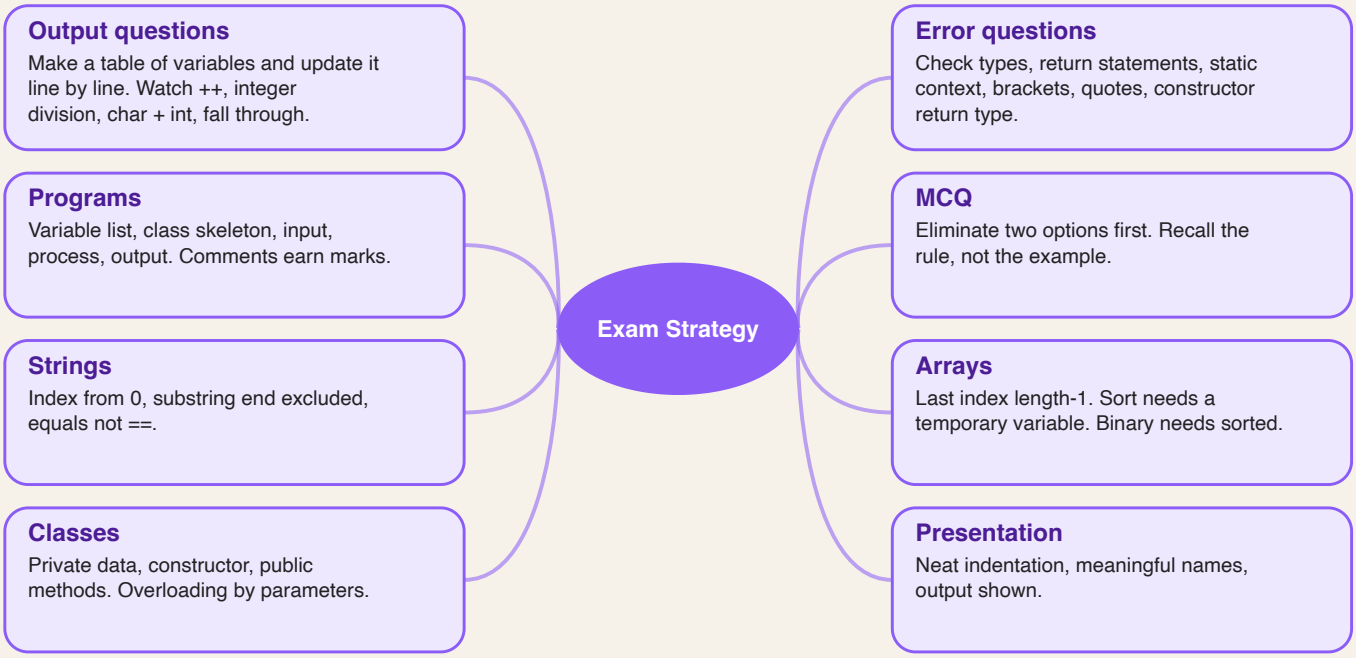
Q	ANS	Q	ANS	Q	ANS	Q	ANS
1	b	6	d	11	b	16	c
2	b	7	b	12	b	17	a
3	b	8	b	13	b	18	b
4	c	9	c	14	a	19	b
5	b	10	c	15	a	20	b

Quick reasons: Q3 is 10/4 = 2 (integer division) then 2 × 2.0 = 4.0. Q4: ceil rounds up. Q8: '$' is neither letter nor digit. Q14: P-r-o-g-r-a-m, so 'm' first appears at index 6. Q15: indexes 2, 3, 4 give "mpu". Q17: 'l' (108) is smaller than 'p' (112), so the result is negative.

Score Yourself

SECTION	MARKS	YOUR TARGET
A: Predict the output	9 questions	8 or more correct
B: Find the error	8 questions	7 or more correct
C: Programs	5 programs	Compiles in your head and the logic is right
D: MCQ	20 questions	18 or more correct

Mind Map Recap



11

BONUS CHAPTER · COMPUTER APPLICATIONS

Lab Companion: Assignments and Marking

The Internal Assessment is worth 100 marks and is entirely practical. You complete at least 20 laboratory assignments over the year. Here is what to build and how it is graded.



IN THIS CHAPTER

- Assignment ideas by topic
- Four marking criteria
- Record-keeping tips

NEW Suggested Assignments by Topic (Minimum 20 in the Year)

TOPIC	PROGRAMS TO PRACTISE
User-defined methods	Pure and impure, static and non-static methods; overloaded methods; programs using data members and member methods through an object
Constructors	Default, parameterised and overloaded constructors, with outputs
Library classes	Output of every wrapper method in the syllabus; check whether a character is uppercase, lowercase or a digit
Encapsulation	Identify local, instance, argument, class, private and public variables
Arrays	Access elements; selection and bubble sort; linear and binary search; double-dimensional arrays (row, column and diagonal sums, matrix display)
String handling	Output of all listed String methods; extract and manipulate characters; palindrome, pig Latin, alphabetical order



KEY POINT

The list is indicative only. Teachers and students are encouraged to create **innovative and original assignments**.

NEW How Each Assignment Is Judged: Four Criteria

CRITERION	WHAT THE TEACHER CHECKS
Class design	A suitable class is used, attributes have the right types, encapsulation is done properly and the interface is well designed
Coding and documentation	Sensible names, proper conditions and loops, error handling, and comment statements
Variable description	A clear list of variables with their purpose and data type
Execution / output	The program runs correctly and you have written the output

One possible way to grade each criterion: excellent, good, fair/acceptable or poor/unacceptable, each given a numeric value, then added up.



SIDEKICK TIP

For every program write the **Variable Description Table** first (name, type, purpose). It earns marks on its own and also makes coding easier.

BOARD ALERT

Do not rely on memorising programs. The teacher evaluates the written work and the practical work for all assignments, so you must be able to explain your code.

NEW A Model Assignment Write-Up

Below is how one assignment could be submitted. The write-up has a title, a problem statement, a variable description table, comments in the code and the output.

Assignment: Palindrome check for a word (String handling)

VARIABLE	DATA TYPE	PURPOSE
s	String	Word entered by the user
rev	String	Reverse of s, built character by character
i	int	Loop index from the last character to the first

```
import java.util.*;
public class LabPalin
{
    // Reads a word and tells whether it is a palindrome
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String s = sc.next();
        String rev = "";                        // reverse of s
        for (int i = s.length() - 1; i ≥ 0; i--)
            rev = rev + s.charAt(i);
        if (s.equalsIgnoreCase(rev))
            System.out.println("Palindrome");
        else
            System.out.println("Not a palindrome");
    }
}
```

```
Input: Radar
-----------------------------
Enter a word: Palindrome
```

NEW Assignment Tracker: 20 in the Year

NO.	TOPIC	PROGRAM IDEA	DONE
1	Revision	Menu driven calculator using switch	
2	Revision	Number pattern with nested loops	
3	Methods	Pure and impure methods in one class	
4	Methods	Static and non-static methods	
5	Methods	Overloaded area calculator	
6	Methods	Objects and data members (student marks)	
7	Constructors	Default and parameterised constructor for a Box	
8	Constructors	Constructor overloading and output	
9	Library classes	Output of all wrapper methods	
10	Library classes	Classify the characters of a string	
11	Encapsulation	Identify the variable kinds in a class	
12	Arrays	Largest, smallest and average of an array	
13	Arrays	Selection sort	
14	Arrays	Bubble sort	
15	Arrays	Linear and binary search	
16	Arrays	Matrix row, column and diagonal sums	
17	Strings	Outputs of every String method	
18	Strings	Extract and modify characters (case swap)	
19	Strings	Palindrome and pig Latin	
20	Strings	Sort a String array alphabetically	



SIDEKICK TIP

Keep your file tidy: one folder per topic, one class per file named after the class. Take a screenshot or print of every output and stick it next to the code in your record.



KEY POINT

Original ideas score highest. A school timetable clash checker, a bill generator for your canteen or a quiz marker are all valid assignments if they use the syllabus concepts.

Done reading?
Now go break the board.



saviours.pro

Sidekick · Computer Applications · ICSE Class 10 · 2027 syllabus edition
A Saviours AI publication. Study smarter with Pre Board 60, mock tests and more on saviours.pro.